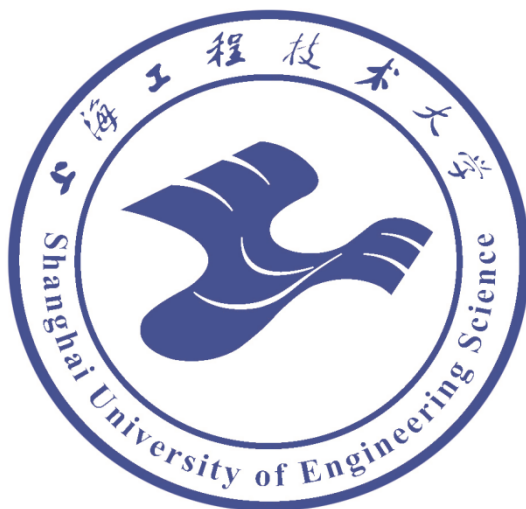


《数据采集与可视化技术》

TapTap 排行榜数据采集分析及可视化



题 目： TapTap 排行榜数据采集分析及可视化

院 系： *****

学 号： *****

姓 名： ***

授课教师： ***

完成时间： 2021 年 1 月 17 日

目录

一、项目内容	3
二、项目步骤	3
(一) pycharm 爬取数据	3
1.数据摘要	3
2.数据分析	3
3.代码分析	5
4.注意事项	6
(二) 使用 datax 导入数据到 mysql.....	6
1.安装 mysql	6
2.在 mysql 中建数据库和表	7
3.配置 json 文件	7
4.运行 datax	8
(三) kettle 清洗数据	9
0.准备工作	9
1.连接数据库	9
2.建立转换	10
3.查看数据库	13
(四) ELK 建立索引及可视化	14
1.配置 elasticsearch.....	14
2.使用 logstash 将数据导入 elasticsearch.....	16
3.使用 kibana 进行数据可视化	18
三、项目数据分析	21
四、项目总结	24
五、附录	26
(一) 软件环境	26
(二) 多线程异步爬虫代码	26
(二) json 文件	39
(三) conf 文件	41

一、项目内容

TapTap 是一个推荐高品质手游的手游分享社区，实时同步全球各大应用市场游戏排行榜，与全球玩家共同交流并发掘高品质手游。本项目目的是了解各大热门手游的具体信息，分析手游玩家热衷的游戏类型等。

本次使用 pycharm 爬取 TapTap 排行榜上的相关榜单，将爬取下来的数据以 csv 格式保存，使用 datax 将源数据导入 mysql 数据库，之后使用 kettle 从 mysql 获取数据进行清洗，再保存到 mysql 数据库中。使用 logstash 将清洗完毕的数据导入 elasticsearch 建立索引，最后使用 kibana 进行可视化操作。

二、项目步骤

（一）pycharm 爬取数据

1. 数据摘要

数据日期：2021 年 1 月 6 日

数据数量：TapTap 安卓游戏榜——热门榜（根据下载量）Top150、热玩榜（根据玩家游戏启动量）Top300；IOS 游戏榜——热门榜（根据下载量）Top120、热玩榜（根据玩家游戏启动量）Top300。（以安卓热玩榜 300 条数据作为范例）

数据类型：游戏排名、游戏名称、游戏厂商、游戏评分、游戏分类、游戏标签、数据（包含安装数量、关注人数、预约人数）。

2. 数据分析

打开 TapTap 排行榜首页可以发现需要爬取的游戏排名、游戏名称、游戏厂商、游戏评分、游戏分类以及游戏标签。排行榜是一个典型的列表+详情结构。

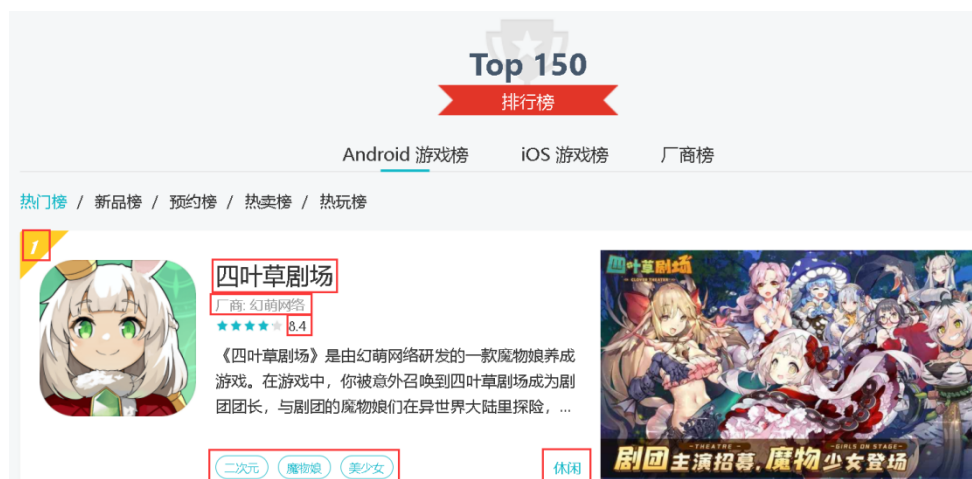


图 2.1.1 Android 热门榜首页

但“数据（包含安装数量、关注人数、预约人数）”需要单击相应的游戏名称或图标、跳转到另一个页面才可以看到。



图 2.1.2 单个游戏介绍界面

将页面拉至底部，可以看到一页只展示 30 条信息，而更多游戏信息的获取需要单击“更多”按钮才会加载下一页的数据，而且网址没有变化。按住 Fn+F12，打开浏览器的开发者工具 Network 中的 XHR，点击“更多”后发现一条新请求，进去就会发现这是查看更多数据的 ajax 异步请求。

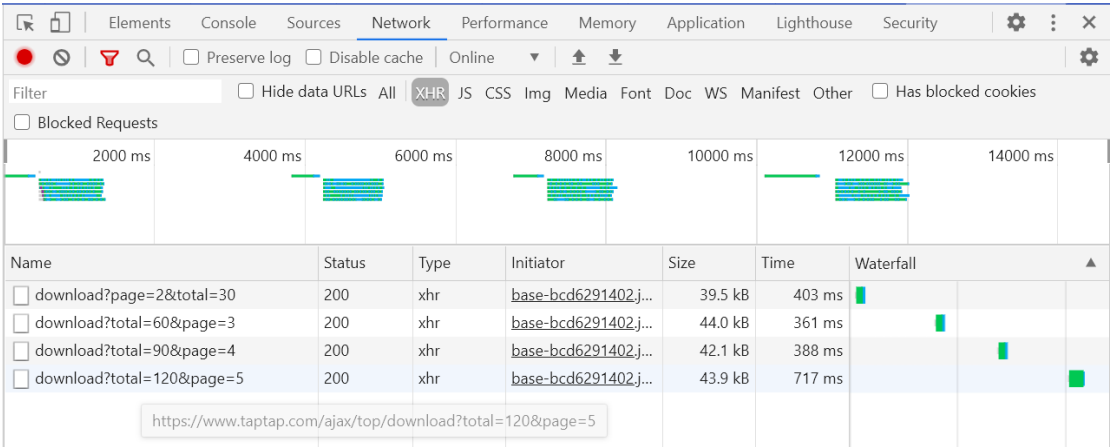


图 2.1.2 ajax 动态加载数据

综上，分析可知需要采用动态网页爬取技术，有以下两种方法：通过浏览器审查元素解析地址；通过 selenium 模拟浏览器抓取。

由于打开 <https://www.taptap.com/ajax/top/download?total=60&page=3> 发现，数据源码并不是规整的 json 格式，且本人现有的 beautifulsoup 技术只能获取游戏名称，所以最终选择方法二，使用 selenium 模拟鼠标点击“更多”的操作来获取其他页面的信息。

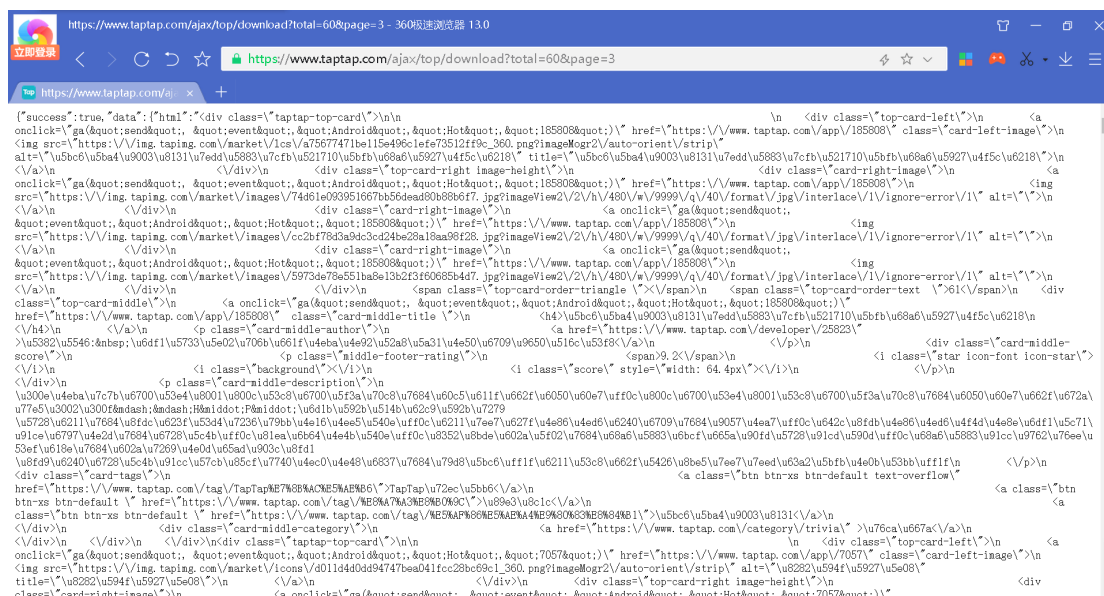


图 2.1.2 其他页面的源码

在 `pycharm` 中新建文件，编写实现代码。因为使用 `selenium` 模拟鼠标点击，一页只新增 30 条数据，所以爬取 300 条数据时就需要点击 9 次“更多”按钮，同时，为了获取“数据（包含安装次数、关注人数、预约人数）”，每一个游戏都对应一个网页、一次鼠标点击操作，所以还需要点击 300 次、开启 300 个新页面。实践发现，单线程爬取 150 条数据是可以成功的，耗时大约 25-40 分钟，但若单线程爬取 300 条数据，经常是爬了一半的数据就会因为网速等原因，导致页面未及时加载而报错找不到元素。因此对源代码进行改进。最终采用多线程技术，最多一次开启 5 个页面，页面 1 只显示前 30 条，页面 2 只显示前 60 条，页面 3 只显示前 90 条，以此类推，5 个爬虫齐头并进。爬虫 1 只抓取 1-30 条数据，爬虫 2 只抓取 31-60 条数据，以此类推，避免后面的爬虫重复获取数据。

3.代码分析

经上述分析，考虑到代码的可读性和封装性，编写了一个函数模拟鼠标点击，即翻页操作；一个函数获取首页（包含翻页之后加载的数据）可以获得的数据，如游戏排名、游戏名称、游戏厂商、游戏评分、游戏分类以及游戏标签；一个函数获取每个游戏对应的“数据（包含安装数量、关注人数、预约人数）”，其中首先要获得跳转页面的链接，之后打开再读取数据，5个页面同时进行。因为是多线程进行，所以最多定义了五个类，每个类里面都重写了 `run()` 方法，获取对应的页面，例如 `getpage3(threading.Thread)` 获取只包含前 90 个游戏的页面。一个

函数 `multi_thread()` 开启多线程，使页面 1、页面 2、页面 3、页面 4、页面 5 同时开启。最后主进程运行 `multi_thread()` 方法，数据获取完毕再保存至列表里，最后写入 csv 文件。

4. 注意事项

- (1) 添加 `webdriver` 的配置，防止因“不安全”而无法打开网页。
- (2) 添加锁 `threading.Lock()`，避免数据未爬完就写入列表。
- (3) 因 300 条的数据分为前 150 条和后 150 条爬取，所以前 150 条数据写入 csv 文件时只需配置条件“`encoding='utf_8_sig', index=False`”，表示按 UTF-8 的格式存储，且不需要索引列。而后 150 条数据写入时，需要增加条件“`mode='a+', header=False`”，设置写入模式为增加且不加列名。

(二) 使用 `datax` 导入数据到 mysql

1. 安装 mysql

使用 `windows+R` 打开命令行窗口，进入 mysql 的 bin 路径，使用命令【`mysqld --install`】安装 mysql。命令【`mysqld --initialize --console`】初始化生成秘钥。

```
D:\Data_acquisition\mysql\mysql-5.7.32-winx64\bin>mysqld --install
Service successfully installed.

D:\Data_acquisition\mysql\mysql-5.7.32-winx64\bin>mysqld --initialize --console
2021-01-13T15:56:50.727568Z 0 [Warning] TIMESTAMP with implicit DEFAULT value is deprecated. Please use --explicit_defaults_for_timestamp server option (see documentation for more details).
2021-01-13T15:56:51.318918Z 0 [Warning] InnoDB: New log files created, LSN=45790
2021-01-13T15:56:51.41449Z 0 [Warning] InnoDB: Creating foreign key constraint system tables.
2021-01-13T15:56:51.523577Z 0 [Warning] No existing UUID has been found, so we assume that this is the first time that this server has been started. Generating a new UUID: f35c6e46-55b7-11eb-abbe-00ffaabbccdd.
2021-01-13T15:56:51.530686Z 0 [Warning] Gtid table is not ready to be used. Table 'mysql.gtid_executed' cannot be opened.
2021-01-13T15:56:53.008909Z 0 [Warning] CA certificate ca.pem is self signed.
2021-01-13T15:56:53.748721Z 1 [Note] A temporary password is generated for root@localhost: Uy,rzA&_c4sP
```

命令【`net start mysql`】启动 mysql 服务。输入命令【`mysql -u root -p`】运行 mysql。初次使用密码为秘钥。

```
D:\Data_acquisition\mysql\mysql-5.7.32-winx64\bin>net start mysql
MySQL 服务正在启动。
MySQL 服务已经启动成功。

D:\Data_acquisition\mysql\mysql-5.7.32-winx64\bin>mysql -u root -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 2
Server version: 5.7.32

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

命令【`alter user 'root'@'localhost' identified by 'root';`】修改密码为“root”，之后使用【`exit`】退出 mysql，重新登录。

```
mysql> alter user 'root'@'localhost' identified by 'root'
-> ;
Query OK, 0 rows affected (0.00 sec)

mysql> exit
Bye

D:\Data_acquisition\mysql\mysql-5.7.32-winx64\bin>mysql -u root -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 3
Server version: 5.7.32 MySQL Community Server (GPL)

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

2.在 mysql 中建数据库和表

```
mysql> create database taptap;
Query OK, 1 row affected (0.00 sec)

mysql> use taptap;
Database changed
mysql> create table android_hotplayed(
-> top int,
-> name varchar(100),
-> firm varchar(100),
-> score varchar(100),
-> category varchar(100),
-> tag varchar(100),
-> statistics varchar(100)
-> )ENGINE=InnoDB DEFAULT CHARSET=utf8;
Query OK, 0 rows affected (0.02 sec)

mysql>
```

3.配置 json 文件

```
1  {
2    "job": {
3      "content": [
4        {
5          "reader": {
6            "name": "txtfilereader",
7            "parameter": {
8              "path": ["E:/TapTap/Android_HotPlayed.csv"],
9              "encoding": "GBK",
10             "fieldDelimiter": ";",
11             "skipHeader": "true",
12             "column": [
13               {
14                 "index": 0,
15                 "type": "string"
16               },
17               {
18                 "index": 1,
19                 "type": "string"
20               },
21               {
22                 "index": 2,
23                 "type": "string"
24               },
25               {
26                 "index": 3,
27                 "type": "string"
28               },
29               {
30                 "index": 4,
```

(全部代码见附录)

4.运行 datax

使用 windows+R 打开命令行窗口, 进入 datax 的 bin 路径, 使用命令【python datax.py D:\Data_acquisition\datax\datax\job\Android_HotPlayed.json】进行数据传输。

因控制台会显示乱码, 直接输入【CHCP 65001】即可解决。

```
Active code page: 65001
D:\Data_acquisition\datax\datax\bin>python datax.py D:\Data_acquisition\datax\datax\job\Android_HotPlayed.json
DataX (DATAX-OPENSOURCE-3.0), From Alibaba !
Copyright (C) 2010-2017, Alibaba Group. All Rights Reserved.

2021-01-15 09:58:59.016 [main] INFO VMInfo - VMInfo# operatingSystem class => sun.management.OperatingSystemImpl
2021-01-15 09:58:59.025 [main] INFO Engine - the machine info =>

    osInfo: Oracle Corporation 1.8 25.271-b09
    jvmInfo:   Windows 10 amd64 10.0
    cpu num:   8

    totalPhysicalMemory: -0.00G
    freePhysicalMemory:  -0.00G
    maxFileDescriptorCount: -1
    currentOpenFileDescriptorCount: -1

    GC Names      [PS MarkSweep, PS Scavenge]

    MEMORY_NAME   allocation_size   init_size
    PS Eden Space  256.00MB         256.00MB
    Code Cache     240.00MB         2.44MB
    Compressed Class Space 1,024.00MB        0.00MB
    PS Survivor Space 42.50MB          42.50MB
    PS Old Gen     683.00MB         683.00MB
    Metaspace      -0.00MB          0.00MB

    MEMORY_NAME   allocation_size   init_size
    PS Eden Space  256.00MB         256.00MB
    Code Cache     240.00MB         2.44MB
    Compressed Class Space 1,024.00MB        0.00MB
    PS Survivor Space 42.50MB          42.50MB
    PS Old Gen     683.00MB         683.00MB
    Metaspace      -0.00MB          0.00MB

2021-01-15 09:59:09.446 [job-0] INFO JobContainer -
任务启动时刻          : 2021-01-15 09:58:59
任务结束时刻          : 2021-01-15 09:59:09
任务总计耗时          : 10s
任务平均流量          : 1.63KB/s
记录写入速度          : 30rec/s
读出记录总数          : 300
读写失败总数          : 0
```

在 mysql 中输入【select * from android_hotplayed;】查看数据。

top	name	firm	score	category	tag	statistics
1	王者荣耀 CN	腾讯	4.8	动作	多人对战, MOBA, 联机	30745684 人安装 34838816 人关注
2	明日方舟 CN	鹰角网络	8	策略	塔防, 二次元, 养成	402925 人安装 770747 人关注
3	使命召唤手游 CN	腾讯	8	动作	第一人称射击, 高画质, 多人对战	7085312 人安装 2569735 人关注
4	妄想山海	腾讯	6.7	角色扮演	开放世界, 高自由度, 山海经	7490505 人安装 3860147 人关注
5	原神 CN	miHoYo	4.4	角色扮演	开放世界, 二次元, 角色扮演	813166 人安装 1177149 人关注
6	和平精英 CN	腾讯	6.9	动作	多人联机, 射击, 第一人称射击	726668 人安装 875002 人关注
7	幻书启世录	网易游戏	4.6	角色扮演	高画质, 二次元, 卡牌	664894 人安装 2304695 人关注
8	阴阳师	网易游戏	7.1	角色扮演	卡牌, 养成, 回合制	2430089 人安装 1757736 人关注
9	光·遇 CN	网易游戏	6	角色扮演	唯美, 治愈, RPG	10262612 人安装 5020882 人关注
10	另一个伊甸: 超越时空的猫 CN	心动网络	9.1	角色扮演	单机, 回合制, RPG	3994929 人安装 5286496 人关注
11	渡魂之刃	腾讯	5.9	动作	ARPG, 动作, Roguelike	3470135 人安装 2201775 人关注
12	天涯明月刀	腾讯	6.7	角色扮演	武侠, 高画质, MMORPG	6885668 人安装 3035596 人关注
13	雷霆英雄 CN	腾讯	8.6	策略	二次元, 美少女, 养成	4160757 人安装 2777269 人关注
14	炉石传说	暴雪	7.2	策略	CCG, 暴雪, 策略	150519 人安装 1494428 人关注
15	火影忍者	腾讯	7	动作	格斗, 动作, ACG	7707859 人安装 3635902 人关注
16	命运-冠位指定 (Fate/Grand Order) CN	网易游戏	7.6	角色扮演	二次元, 卡牌养成, 卡牌	17700602 人安装 8233943 人关注
17	决战! 平安京 CN	网易游戏	6.5	角色扮演	MOBA, 联机, 多人	13807969 人安装 4224514 人关注
18	战双帕弥什 CN	库洛游戏	7.2	角色扮演	动作, 高画质, 二次元	572075 人安装 841967 人关注
19	云顶之弈 (游戏下载)	腾讯	8.6	角色扮演	ARPG, G3, MMORPG	1950497 人安装 1394205 人关注
20	自走棋 (游戏下载)	腾讯	9.3	冒险	卡牌, 联机, Roguelike	7190355 人安装 1083948 人关注
21	元梦之星	腾讯	9.6	动作	Roguelike, 地牢, 像素	10657832 人安装 3960130 人关注
22	崩坏3	miHoYo	6.8	动作	动作, 二次元, 高画质	1004757 人安装 653481 人关注
23	牧场物语	腾讯	6.7	角色扮演	武侠, 高自由度, 开放世界	6641517 人安装 10220764 人关注
24	英雄联盟 CN	腾讯	4.4	角色扮演	冒险, 策略, 放置	1441830 人安装 9040724 人关注
25	穿越火线: 枪战王者	腾讯	5.9	动作	第一人称射击, 吃鸡, 联机	1241859 人安装 1761916 人关注
26	江南百景图 CN	网易游戏	7.1	模拟	建设, 经营, 放置	2453028 人安装 1822327 人关注
27	诸神皇冠	微士	8.1	策略	单机, SRPG, 战棋	9029066 人安装 17573683 人关注

273	像素射击 - Cops N Robbers (测试版)	厂商: 巨量引擎	7.1	动作	像素, 枪战, 联机	2983742 人关注	1431571 人关注
274	大航海探险物语	厂商: 识君	8	模拟	开罗, 像素, 联网	855290 人关注	477334 人关注
275	一代掌门 (测试版)	厂商: 悠易网络	6.8	模拟	经营, 模拟, 策略	643789 人关注	402987 人关注
276	物种起源	厂商: 起源工作室	8.8	策略	放置, 科幻, 挂机	2382096 人关注	
277	长空羽衣 CN	厂商: 腾讯	7.2	角色扮演	捏脸, 换装, 3D	510792 人关注	381616 人关注
278	同步音律	厂商: 廊坊同步音律信息科技有限公司	9.3	音乐	节奏, 音乐, 二次元	199217 人关注	144364 人关注
279	航海王: 燃烧意志	厂商: 麦九游信息技术有限公司	5.1	角色扮演	动漫, ACG, 回合制	233499 人关注	
280	口袋妖怪物语2	厂商: 识君	9.2	模拟	单机, 开罗, 像素	2288010 人关注	2398310 人关注
281	功夫英雄 (测试版) CN	厂商: Ohayo	6.8	动作	格斗, 开罗, 休闲	1902 人关注	
282	螺旋圆舞曲	厂商: 广州天梯网络科技有限公司	6.6	角色扮演	换装, 女性向, 养成	213896 人关注	585923 人关注
283	女将星录	厂商: 银河网络	7.4	角色扮演	养成, 挂机, 放置	1094746 人关注	1899604 人关注
284	女将星录 (测试版) CN	厂商: 银河网络	7.4	角色扮演	养成, 挂机, 放置	638270 人关注	481113 人关注
285	女将星录 (测试版) CN	厂商: 银河网络	7.4	角色扮演	养成, 挂机, 放置	496504 人关注	1552330 人关注
286	女将星录 (测试版) CN	厂商: 银河网络	7.4	角色扮演	养成, 挂机, 放置	21553 人关注	27618 人关注
287	战略竞技场 CN	厂商: 腾讯	5.1	策略	自走棋, 对战, 竞技	136911 人关注	50596 人关注
288	战略竞技场 CN	厂商: 深圳市创梦天地科技有限公司	6.3	策略	自走棋, 对战, 竞技	146302 人关注	495544 人关注
289	战略竞技场 CN	厂商: 基丰永通 (北京) 科技股份有限公司	7.4	策略	自走棋, 对战, 竞技	470846 人关注	424835 人关注
290	战略竞技场 CN	厂商: 腾讯	4.3	策略	自走棋, 对战, 竞技	496532 人关注	384768 人关注
291	精英竞技场 CN	厂商: 海南一五一十网络科技有限公司	3.4	角色扮演	角色扮演, MMO, 像素	498890 人关注	100112 人关注
292	精英竞技场 CN	厂商: METER	6.3	角色扮演	角色扮演, MMO, 像素	2036289 人关注	1185014 人关注
293	精英竞技场 CN	厂商: 腾讯	2.8	角色扮演	角色扮演, MMO, 像素	135488 人关注	
294	精英竞技场 CN	厂商: 自源计划工作室	6.4	角色扮演	角色扮演, MMO, 像素	234259 人关注	306340 人关注
295	精英竞技场 CN	厂商: 北京乐动卓越科技有限公司	8.9	角色扮演	角色扮演, MMO, 像素	427961 人关注	443245 人关注
296	精英竞技场 CN	厂商: 群象游戏	7.4	角色扮演	角色扮演, MMO, 像素	664718 人关注	564895 人关注
297	精英竞技场 CN	厂商: 群象游戏	7.4	角色扮演	角色扮演, MMO, 像素	141661 人关注	308315 人关注
298	精英竞技场 CN	厂商: 群象游戏	7.4	角色扮演	角色扮演, MMO, 像素	2358972 人关注	1812247 人关注
299	精英竞技场 CN	厂商: 群象游戏	7.4	角色扮演	角色扮演, MMO, 像素	1328809 人关注	1913039 人关注
300	精英竞技场 CN	厂商: 群象游戏	7.3	角色扮演	角色扮演, MMO, 像素	232192 人关注	172106 人关注

(三) kettle 清洗数据

0.准备工作

在 mysql 中建好清洗完毕的数据要导入的表。

```
mysql> create table android300(
-> top int,
-> name varchar(100),
-> firm varchar(100),
-> score float,
-> category varchar(100),
-> tag1 varchar(100),
-> tag2 varchar(100),
-> ag3 varchar(100),
-> download int,
-> concern int
-> )ENGINE=InnoDB DEFAULT CHARSET=utf8;
Query OK, 0 rows affected (0.03 sec)
```

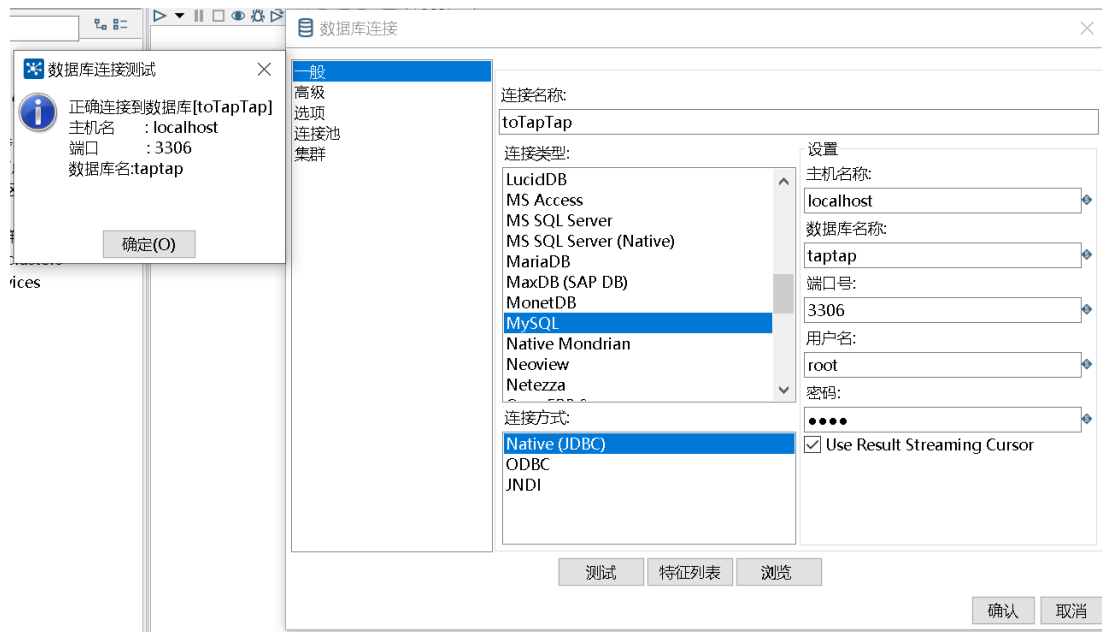
1.连接数据库

打开 kettle 目录下的 spoon.bat 文件。

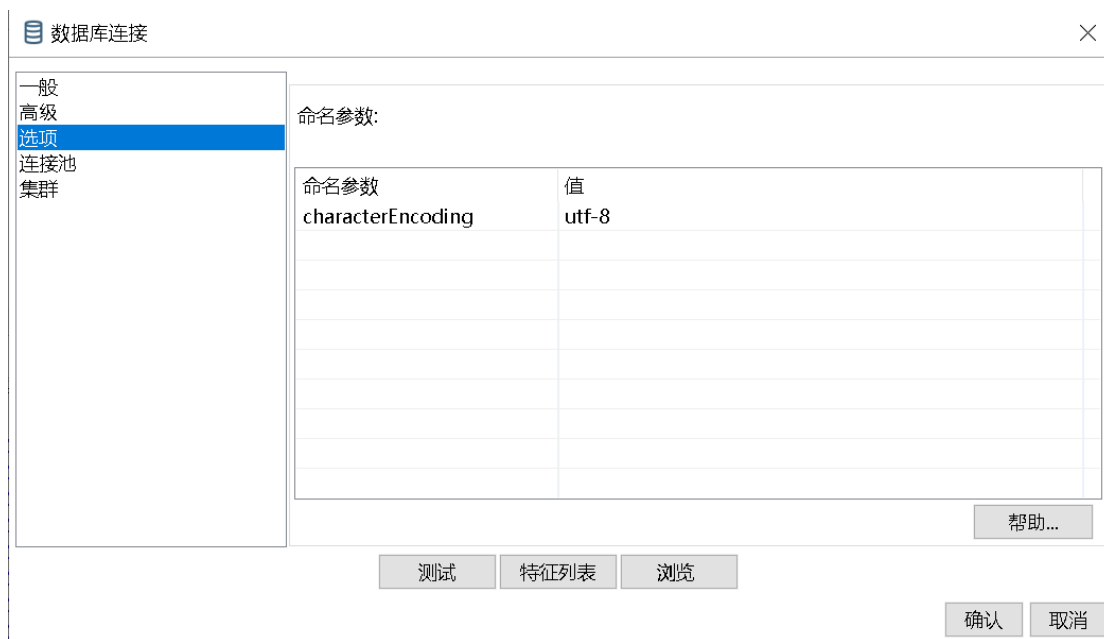
« Data_acquisition » kettle » pdi-ce-7.1.0.0-12 » data-integration					搜索"data-integration"
名称	修改日期	类型	大小		
PentahoDataIntegration_OSS_Licenses.html	2017/5/16 16:58	360 Chrome HTML ...	9,904 KB		
purge-utility.bat	2017/5/16 20:08	Windows 批处理文件	1 KB		
purge-utility.sh	2017/5/16 20:08	SH 文件	1 KB		
README.txt	2017/5/16 20:08	文本文档	2 KB		
runSamples.bat	2017/5/16 20:08	Windows 批处理文件	1 KB		
runSamples.sh	2017/5/16 20:08	SH 文件	1 KB		
set-pentaho-env.bat	2017/5/16 20:08	Windows 批处理文件	5 KB		
set-pentaho-env.sh	2017/5/16 20:08	SH 文件	4 KB		
Spark-app-builder.bat	2017/5/16 20:08	Windows 批处理文件	1 KB		
spark-app-builder.sh	2017/5/16 20:08	SH 文件	1 KB		
Spoon.bat	2020/12/16 17:28	Windows 批处理文件	4 KB		
spoon.command	2017/5/16 20:08	COMMAND 文件	1 KB		
spoon.ico	2017/5/16 20:08	ICO 图片文件	362 KB		
spoon.png	2017/5/16 20:08	PNG 图片文件	2 KB		
spoon.sh	2017/5/16 20:08	SH 文件	7 KB		
SpoonConsole.bat	2017/5/16 20:08	Windows 批处理文件	1 KB		
SpoonDebug.bat	2017/5/16 20:08	Windows 批处理文件	2 KB		

新建一个“转换”。配置 DB, mysql 的端口号为 3306, 填写完毕可以点击

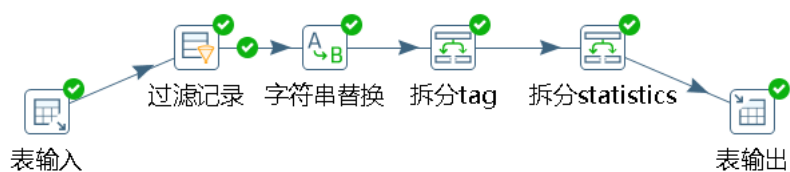
“测试”，查看数据库是否连接成功。



在“选项”页面，添加 characterEncoding utf-8，以此避免出现数据迁移时中文乱码的情况。



2. 建立转换



表输入：从 mysql 导入数据

表输入

步骤名称: 表输入

数据库连接: toTapTap [编辑... 新建... Wizard...]

SQL: `SELECT * FROM android_hotplayed;` [获取SQL查询语句...]

行1 列0

允许简易转换 ☐

替换 SQL 语句里的变 ☐

从步骤插入数据 [下拉菜单]

执行每一行? ☐

记录数量限制: 0

[Help] [确定(O)] [预览(P)] [取消(C)]

过滤记录：只保留“数据”中含有“安装”或者“购买”字段的数据。

过滤记录

步骤名称: 过滤记录

发送true数据给步骤: 字符串替换

发送false数据给步: [下拉菜单]

条件:

- statistics CONTAINS [安装]
- OR
- statistics CONTAINS [购买]

[Help] [确定(O)] [取消(C)]

字符串替换：去除多余字段以及不必要的空格

字符串替换

步骤名称: 字符串替换

#	输入流字段	输出流字段	使用正则表达式	搜索	使用...替换	设置为空串?	使用字段值替换	整个单词匹配	大小写敏感
1	firm		否	厂商:		否		否	否
2	statistics		否	人购买		否		否	否
3	statistics		否	人关注		否		否	否
4	statistics		否	人安装		否		否	否
5	name		否	CN		否		否	否

[Help] [确定(O)] [获取字段(G)] [取消(C)]

拆分字段：将 tag 分割开，例：宠物,治愈,放置 --> 宠物 治愈 放置。

拆分字段

步骤名称: 拆分tag

需要拆分的字段: tag

分隔符: ,

Enclosure:

字段

#	新的字段	ID	移除ID?	类型	长度	精度	格式	分组符号	小数点符号	货币符号	Nullif	缺省	去除空格类型
1	tag1		N	String									不去掉空格
2	tag2		N	String									不去掉空格
3	tag3		N	String									不去掉空格

Help 确定(O) 取消(C)

将 statistics 分隔成下载次数 download 和关注人数 concern。

拆分字段

步骤名称: 拆分statistics

需要拆分的字段:

分隔符:

Enclosure:

字段

#	新的字段	ID	移除ID?	类型	长度	精度	格式	分组符号	小数点符号	货币符号	Nullif	缺省	去除空格类型
1	download		N	Integer									不去掉空格
2	concern		N	Integer									不去掉空格

Help 确定(O) 取消(C)

表输出：清洗好的数据存入 mysql 的 taptap 数据库中的表 android300。

表输出

步骤名称: 表输出

数据库连接: toTapTap 编辑... 新建... Wizard...

目标模式: 浏览(B)...

目标表: android300 浏览(B)...

提交记录数量: 1000

裁剪表: ☐

忽略插入错误: ☐

指定数据库字段: ☐

主选项 数据库字段

表分区数据: ☐

分区字段:

每个月分区数据: ☒

每天分区数据: ☐

使用批量插入: ☒

表名定义在一个字段里?: ☐

包含表名的字段:

存储表名字段: ☒

返回一个自动产生的关键字: ☐

自动产生的关键字的字段名称:

Help 确定(O) 取消(C) SQL

运行这个转换，执行结果如下。

执行结果

执行历史 | 日志 | 步骤度量 | 性能图 | Metrics | Preview data

2021/01/16 23:51:25 - Spoon - Using legacy execution engine
2021/01/16 23:51:25 - Spoon - 转换已经打开.
2021/01/16 23:51:25 - Spoon - 正在打开转换 [1]...
2021/01/16 23:51:25 - Spoon - 开始执行转换.
2021/01/16 23:51:25 - 1 - 为了转换解除补丁开始 [1]
2021/01/16 23:51:25 - 表输出.0 - Connected to database [toTapTap] (commit=1000)
2021/01/16 23:51:25 - 表输入.0 - Finished reading query, closing connection.
2021/01/16 23:51:25 - 过滤记录.0 - 完成处理 (I=0, O=0, R=300, W=249, U=0, E=0)
2021/01/16 23:51:25 - 表输入.0 - 完成处理 (I=300, O=0, R=0, W=300, U=0, E=0)
2021/01/16 23:51:25 - 字符串替换.0 - 完成处理 (I=0, O=0, R=249, W=249, U=0, E=0)
2021/01/16 23:51:25 - 拆分tag.0 - 完成处理 (I=0, O=0, R=249, W=249, U=0, E=0)
2021/01/16 23:51:25 - 拆分statistics.0 - 完成处理 (I=0, O=0, R=249, W=249, U=0, E=0)
2021/01/16 23:51:25 - 表输出.0 - 完成处理 (I=0, O=249, R=249, W=249, U=0, E=0)
2021/01/16 23:51:25 - Spoon - 转换完成!!

共读入 300 条数据，输出 249 条数据。

执行结果

#	步骤名称	复制的记录行数	读	写	输入	输出	更新	拒绝	错误	激活	时间	速度 (条记录/秒)	Pri/in/out
1	表输入	0	0	300	300	0	0	0	0	已完成	0.0s	15,789	-
2	过滤记录	0	300	249	0	0	0	0	0	已完成	0.0s	16,667	-
3	字符串替换	0	249	249	0	0	0	0	0	已完成	0.0s	9,960	-
4	拆分tag	0	249	249	0	0	0	0	0	已完成	0.0s	8,586	-
5	拆分statistics	0	249	249	0	0	0	0	0	已完成	0.0s	6,917	-
6	表输出	0	249	249	0	249	0	0	0	已完成	0.1s	2,110	-

3.查看数据库

top	name	firm	score	category	tag1	tag2	tag3	download	concern
1	王者荣耀	腾讯	4.8	动作	多人对战	MOBA	联机	30745584	34838816
2	明日方舟	鹰角网络	8	策略	塔防	二次元	养成	402925	776747
3	使命召唤手游	腾讯	8	动作	第一人称射击	高画质	多人对战	7085312	2589735
4	英雄联盟	腾讯	6.7	角色扮演	开放世界	高自由度	山海经	7490505	3860147
5	原神	miHoYo	4.4	角色扮演	开放世界	二次元	角色扮演	813166	1177149
6	和平精英	腾讯	6.9	动作	多人联机	射击	第一人称射击	726668	873002
7	幻书启世录	网易游戏	4.6	动作	卡牌	二次元	回合制	364894	2304595
8	阴阳师	网易游戏	7.1	角色扮演	养成	回合制	回合制	2430038	1787738
9	另一个伊甸：超越时空的猫	心动网络	6	策略	唯美	治愈	RPG	10262612	5020882
10	魔渊之刃	网易游戏	9.1	动作	单机	回合制	RPG	3994929	5286496
11	天涯明月刀	腾讯	5.8	动作	ARPG	动作	Roguelike	3478135	2201775
12	碧蓝航线	腾讯	8.7	策略	二次元	高画质	养成	6835668	3035896
13	炉石传说	Blizzard Entertainment, Inc.	8.6	策略	CCC	养成	养成	4160757	2777269
14	火影忍者	腾讯	7.2	策略	格斗	ACG	策略	150519	1494428
15	命运-冠位指定 (Rate/Grand Order)	哔哩哔哩游戏	7	动作	格斗	ACG	养成	7707859	3625902
16	奥义！平安京	网易游戏	7.6	角色扮演	二次元	卡牌	养成	17700602	8233943
17	战双帕弥什	网易游戏	6.5	角色扮演	MOBA	联机	多人	15807969	4224514
18	天谕 (预下载)	网易游戏	7.2	角色扮演	动作	高画质	二次元	572075	841967
19	月圆之夜	网易游戏	8.6	角色扮演	幻想	3D	MMORPG	1950497	1994205
20	崩坏3	米哈游工作室	9.3	冒险	卡牌	单机	Roguelike	7190355	1082949
21	崩坏学园2	米哈游	9.6	动作	Roguelike	养成	养成	10657632	3960130
22	我的侠客	miHoYo	6.8	动作	动作	二次元	高画质	1004757	653481
23	最强蜗牛	极光计划	6.7	角色扮演	武侠	高自由度	开放世界	6641517	10220764
24	穿越火线-枪战王者	腾讯	4.4	角色扮演	冒险	射击	多人	1441380	9040724
25	江南百景图	腾讯	5.9	动作	多人	养成	养成	1241859	1761916
26	诸神皇冠	腾讯	7.1	模拟	建设	经营	经营	2453028	1822327
27	QQ飞车	腾讯	8.1	策略	单机	SRPG	养成	90290866	17573683
28	人类跌落梦境	腾讯	7	冒险	竞速	养成	养成	1629472	1537318
29	公主连结Re:Dive	哔哩哔哩游戏	6.6	角色扮演	二次元	卡牌	养成	472032	403968
30	万象物语	龙洲网络	8.5	角色扮演	养成	RPG	养成	559267	1460173
31	英雄联盟	腾讯	7.6	动作	射击	吃鸡	多人	105749	171482
32	英雄联盟	腾讯	6.4	角色扮演	射击	吃鸡	多人	6628924	8105603
33	英雄联盟	腾讯	8.9	卡牌	卡牌	卡牌	卡牌	815071	477068
34	英雄联盟	腾讯	9	动作	单机	高画质	动作	17638925	6371527
35	英雄联盟	腾讯	7.2	角色扮演	高自由度	MMORPG	养成	230755	302794
36	英雄联盟	腾讯	7.6	角色扮演	养成	养成	养成	550109	1127021
37	英雄联盟	腾讯	6.4	角色扮演	养成	养成	养成	141262	289466
38	英雄联盟	腾讯	8.1	角色扮演	养成	养成	养成	1377940	1885716
39	英雄联盟	腾讯	8.5	音乐	音乐	音乐	音乐	484876	336174
40	英雄联盟	腾讯	8.6	音乐	音乐	音乐	音乐	1224598	656804
41	英雄联盟	腾讯	5.6	角色扮演	ARPG	ACG	二次元	3411693	1692220

250	管理员: 命令提示符 - mysql -u root -p	西山居世游	5.0	角色扮演	像素	回合制	角色扮演	424427	517205
251	全民英雄	绵阳普加科技有限责任公司	8.5	益智	益智	休闲	单机	45049	32636
252	新笑傲江湖	完美世界	4.5	角色扮演	武侠	角色扮演	MMO	1474844	955721
254	旅行串串	Kunpo	9.5	休闲	放置	经营	美食	266596	152756
255	遇见泡芙	网易游戏	6.6	角色扮演	恋爱	养成	乙女	27583	110179
258	水生物	边锋工作室	8.7	模拟	真实模拟	养成	高自由度	514785	286799
262	功夫奇遇记 (测试版)	Ohayoo	7.2	冒险	Roguelike	独立游戏	动作	122793	248770
263	仙侠第一放置 (网络版)	撸撸工作室	8.3	角色扮演	放置	修仙	挂机	101410	74153
264	人生模拟器: 中国式人生 (测试版)	Kiwigames	4.5	模拟	模拟	养成	文字	208951	314866
265	奇迹之燕 (测试版)	地心引力工作室	8.4	动作	像素	Roguelike	单机	510126	969806
266	不一样传说 (测试版)	芥菜子游戏	7.6	冒险	角色扮演	养成	单机	554492	463347
267	飞跃星球	重庆星漫科技有限公司	4.5	模拟	宇宙	文字游戏	TapTap独家	2164199	1526477
269	圣斗士星矢 (腾讯)	腾讯	6.2	角色扮演	回合制	卡牌	卡牌	665465	475499
270	四色世界	七创社	6.2	角色扮演	养成	养成	策略	17537	11939
271	永不言弃!!	东品游戏	7.1	音乐	虐心	手残	节奏	1946148	1135301
272	战争艺术	朝夕光年	6.7	策略	高画质	Steam移植	战争	725682	571587
273	像素射击 - Cops N Robbers (测试版)	巨量引擎	7.1	动作	像素	枪战	联机	2983742	1431571
274	大航海冒险物语	识君	6.8	模拟	开罗	像素	联网	355290	477334
275	一体掌门 (测试版)	感趣网络	6.8	模拟	经营	模拟	策略	643789	402987
277	云裳羽衣	腾讯	7.2	角色扮演	捏脸	换装	3D	510792	381616
278	同步音律	廊坊同步音律信息科技有限公司	9.3	音乐	洛天依	音游	二次元	199217	144364
280	口袋学院物语2	识君	9.2	模拟	养成	养成	养成	2288010	2396310
282	模拟经营	广州天梯网络科技有限公司	4.5	角色扮演	养成	养成	养成	213896	585923
283	女神星球	银月网络	7.4	角色扮演	养成	养成	养成	1094746	1899604
284	字走三国 (测试版)	零图游戏	9.1	桌面和棋类	单机	三国	文字	638270	481113
285	警察大师 (测试版)	腾讯	8.1	休闲	音乐	冒险	冒险	485504	1552530
286	星际计划	腾讯	9.6	冒险	国风	单机	单机	21553	27618
287	战歌竞技场	腾讯	5.1	策略	自走棋	对战	竞技	136911	50596
288	地獄跑酷	深圳市创梦天地科技有限公司	6.3	冒险	跑酷	单机	免费	146302	499544
289	邮差骑士	嘉丰水道 (北京) 科技股份有限公司	7.4	角色扮演	单机	养成	角色扮演	470846	424635
290	人之子	海南一五一十网络科技有限公司	4.5	动作	国风	格斗	格斗	498332	394768
291	精英进阶进化	METER	3.4	角色扮演	角色扮演	MMO	像素	499890	100112
292	奥特曼系列OL	启源计划工作室	6.3	角色扮演	肉肉	回合制	回合制	2036289	1185014
294	总攻女神	启源计划工作室	6.4	卡牌	卡牌	养成	回合制	234259	306340
295	航海日记	北京乐动卓越科技有限公司	8.9	冒险	模拟	养成	养成	427961	443245
296	逆刃客 (测试版)	群集游戏	7.4	文字	武侠	修仙	MUD	664718	564895
297	遇见	豆游个人工作室	7.9	角色扮演	开放世界	角色扮演	冒险	141661	308315
298	我的奶茶屋 (测试版)	章鱼工作室	8.3	模拟	经营	休闲	可爱	2958972	1812247
299	闪烁之光	尚游网络	4.6	角色扮演	放置	回合制	卡牌	1328609	1913039
300	万龙指令	哔哩哔哩游戏	7.5	卡牌	二次元	养成	卡牌养成	232192	172106

(四) ELK 建立索引及可视化

1. 配置 elasticsearch

打开 elasticsearch 的 config 文件夹下的 elasticsearch.yml 文件，修改配置。

```

elasticsearch.yml - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

# ----- Cluster -----
#
# Use a descriptive name for your cluster:
#
cluster.name: yyqx
#
# ----- Node -----
#
# Use a descriptive name for the node:
#
node.name: node-1128
#
# Add custom attributes to the node:
#
#node.attr.rack: r1
#
node.master: true
#
# ----- Paths -----
#
# Path to directory where to store the data (separate multiple locations by comma):
#

```

双击 elasticsearch 的 bin 目录下的 elasticsearch.bat 执行。

```
CA:\WINDOWS\system32\cmd.exe
[2021-01-17T00:20:15.881][INFO ][o.e.n.Node ] [node-1128] starting ...
[2021-01-17T00:20:19.147][INFO ][o.e.t.TransportService] [node-1128] publish_address {127.0.0.1:9300}, bound_addresses {127.0.0.1:9300}, {[::]:9300}
[2021-01-17T00:20:19.551][WARN ][o.e.b.BootstrapChecks] [node-1128] the default discovery settings are unsuitable for production use; at least one of [discovery.seed_hosts, discovery.seed_providers, cluster.initial_master_nodes] must be configured
[2021-01-17T00:20:19.553][INFO ][o.e.c.c.Coordinator] [node-1128] cluster UUID [8YcQmBYKSQ-HYMB5IdD43Q]
[2021-01-17T00:20:19.573][INFO ][o.e.c.c.ClusterBootstrapService] [node-1128] no discovery configuration found, will perform best-effort cluster bootstrapping after [3s] unless existing master is discovered
[2021-01-17T00:20:19.710][INFO ][o.e.c.s.MasterService] [node-1128] elected-as-master ([1] nodes joined)[{node-1128}{yxUUs4r9QQ2587iR_ZaEqA}{w4IgL9K9Q_qqiUW-QUchdQ}{127.0.0.1}{127.0.0.1:9300}{dilmrt}{ml.machine_memory=8494747648, xpack.installed=true, transform.node=true, ml.max_open_jobs=20} elect leader, _BECOME_MASTER_TASK_, _FINISH_ELECTION_, term: 12, version: 183, delta: master node changed (previous [], current [{node-1128}{yxUUs4r9QQ2587iR_ZaEqA}{w4IgL9K9Q_qqiUW-QUchdQ}{127.0.0.1}{127.0.0.1:9300}{dilmrt}{ml.machine_memory=8494747648, xpack.installed=true, transform.node=true, ml.max_open_jobs=20}]]], term: 12, version: 183, reason: Publication{term=12, version=183}
[2021-01-17T00:20:20.184][INFO ][o.e.l.LicenseService] [node-1128] license [f79b577e-7941-4633-ace3-b69d681722ae] mode [basic] - valid
[2021-01-17T00:20:20.187][INFO ][o.e.x.s.s.SecurityStatusChangeListener] [node-1128] Active license is now [BASIC]; Security is disabled
[2021-01-17T00:20:20.199][INFO ][o.e.g.GatewayService] [node-1128] recovered [8] indices into cluster state
[2021-01-17T00:20:21.308][INFO ][o.e.c.r.a.AllocationService] [node-1128] Cluster health status changed from [RED] to [YELLOW] (reason: [shards started [{.kibana_task_manager_1}[0]]]).
[2021-01-17T00:20:21.775][INFO ][o.e.h.AbstractHttpServerTransport] [node-1128] publish_address {127.0.0.1:9200}, bound_addresses {127.0.0.1:9200}, {[::]:9200}
[2021-01-17T00:20:21.776][INFO ][o.e.n.Node ] [node-1128] started
```

打开浏览器，测试一下。

```
127.0.0.1:9200 - 360极速浏览器 13.0
立即登录 < > C D ☆ http://127.0.0.1:9200/
127.0.0.1:9200 x +
{
  "name": "node-1128",
  "cluster_name": "yyqx",
  "cluster_uuid": "8YcQmBYKSQ-HYMB5IdD43Q",
  "version": {
    "number": "7.9.3",
    "build_flavor": "default",
    "build_type": "zip",
    "build_hash": "c4138e51121ef06a6404866cddc601906fe5c868",
    "build_date": "2020-10-16T10:36:16.141335Z",
    "build_snapshot": false,
    "lucene_version": "8.6.2",
    "minimum_wire_compatibility_version": "6.8.0",
    "minimum_index_compatibility_version": "6.0.0-beta1"
  },
  "tagline": "You Know, for Search"
}
```

安装 node 环境和 grunt，安装完成后，使用命令查看版本号。

```
管理员: 命令提示符
Microsoft Windows [版本 10.0.18363.1316]
(c) 2019 Microsoft Corporation。保留所有权利。

C:\WINDOWS\system32>node -v
v14.15.3

C:\WINDOWS\system32>grunt -version
grunt-cli v1.3.2

C:\WINDOWS\system32>
```

下载 head 插件，进入 head 文件夹下执行命令：npm install (安装 pathomjs)。

```
D:\Data acquisition\relatedToElastic\elasticsearch-head-master\elasticsearch-head-master>npm install
npm WARN elasticsearch-head@0.0.0 license should be a valid SPDX license expression
npm WARN optional SKIPPING OPTIONAL DEPENDENCY: fsevents@1.2.13 (node_modules/fsevents):
npm WARN notsup SKIPPING OPTIONAL DEPENDENCY: Unsupported platform for fsevents@1.2.13: wanted {"os":"darwin","arch":"any"} (current: {"os":"win32","arch":"x64"})

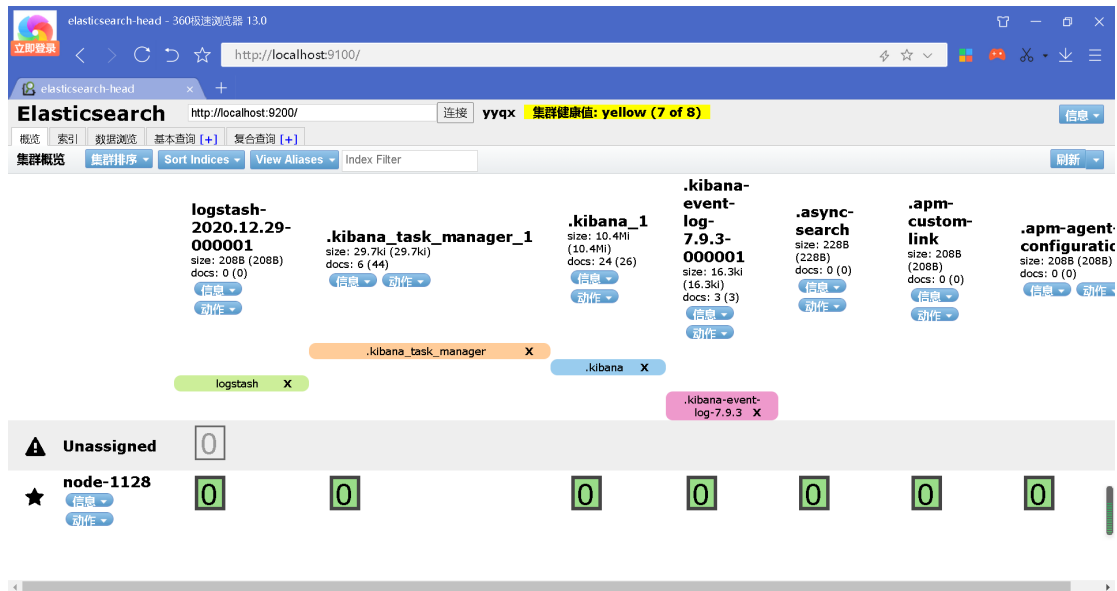
up to date in 13.931s

8 packages are looking for funding
run npm fund for details
```

使用命令 npm start 启用服务。

```
D:\Data_acquisition\relatedToElastic\elasticsearch-head-master\elasticsearch-head-master>npm start
> elasticsearch-head@0.0.0 start D:\Data_acquisition\relatedToElastic\elasticsearch-head-master\elasticsearch-head-master
> grunt server
Running "connect:server" (connect) task
Waiting forever...
Started connect web server on http://localhost:9100
```

使用地址：localhost:9100 访问，出现如下截图，说明 head 安装成功。



2.使用 logstash 将数据导入 elasticsearch

在 logstash 的 bin 目录下新建配置文件 jdbc.conf 并在命令行窗口运行。因为经过 kettle 清洗的数据已经十分标准，所以配置文件只含 input 和 output 模块。

```
"jdbc.conf - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
input {
  stdin {
  }
}
jdbc {
  jdbc_connection_string => "jdbc:mysql://localhost:3306/taptap?characterEncoding=UTF-8&useSSL=false&autoReconnect=true"
  jdbc_user => "root"
  jdbc_password => "root"
  jdbc_driver_library => "D:\Data_acquisition\mysql\mysql-connector-java-5.1.38.jar"
  jdbc_driver_class => "com.mysql.jdbc.Driver"
  jdbc_default_timezone => "Asia/Shanghai"
  statement => "SELECT * FROM android300"
}
}

output {
  elasticsearch {
    hosts => ["127.0.0.1:9200"]
    index => "android_hotplayed_index"
    document_type => "android_hotplayed_type"
    template_overwrite => true
  }
  stdout {
    codec => "rubydebug"
  }
}
}
```

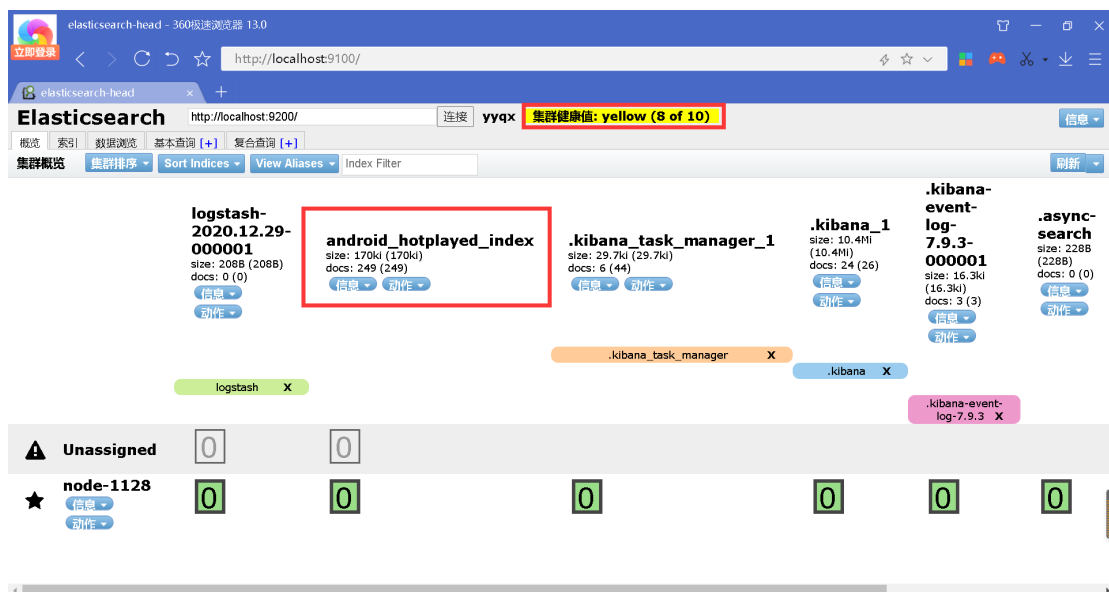
在 logstash 的 bin 路径下，通过 jdbc.conf 配置文件启动 logstash。

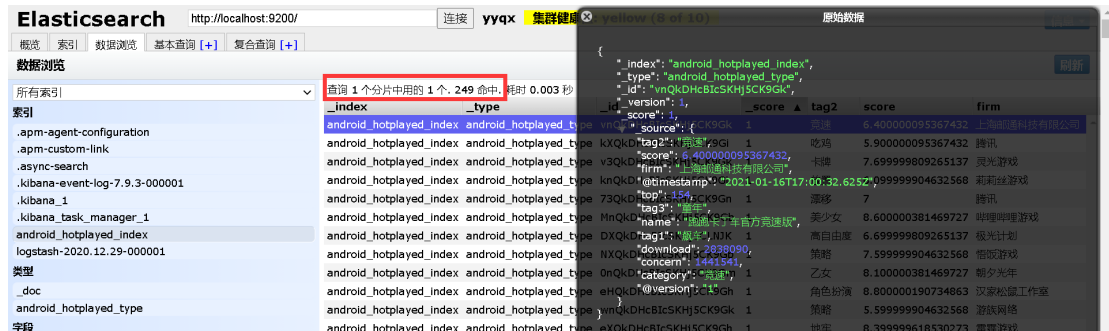

```
管理员: 命令提示符 - logstash -f jdbc.conf
D:\Data_acquisition\logstash-7.9.3\bin>logstash -f jdbc.conf
Sending Logstash logs to D:\Data_acquisition\logstash-7.9.3\logs which is now configured via log4j2.properties
[2021-01-17T01:00:24,054][INFO ][logstash.runner] Starting Logstash {"logstash.version"=>"7.9.3", "jruby.version"=>"jruby 9.2.13.0 (2.5.7) 2020-08-03 9a89e94bec Java HotSpot(TM) 64-Bit Server VM 25.271-b09 on 1.8.0_271-b09 +indy +jit [mswin32-x86_64]"}
[2021-01-17T01:00:24,444][WARN ][logstash.config.source.multilocal] Ignoring the 'pipelines.yml' file because modules or command line options are specified
[2021-01-17T01:00:26,602][INFO ][org.reflections.Reflections] Reflections took 63 ms to scan 1 urls, producing 22 keys and 45 values
[2021-01-17T01:00:27,191][WARN ][logstash.outputs.elasticsearch] You are using a deprecated config setting "document_type" set in elasticsearch. Deprecated settings will continue to work, but are scheduled for removal from logstash in the future. Document types are being deprecated in Elasticsearch 6.0, and removed entirely in 7.0. You should avoid this feature. If you have any questions about this, please visit the #logstash channel on freenode irc. {:name=>"document_type", :plugin=><LogStash::Outputs::ElasticSearch::Index>"android_hotplayed_index", id=>"43dcb6aa53e53ec470744d79b289d1d1fb8840f42fdd5f523b2a8d80ad306da4", hosts=>["http://127.0.0.1:9200"], template_overwrite=>true, document_type=>"android_hotplayed_type", enable_metric=>true, codec=><LogStash::Codecs::Plain id=>"plain_Ga30bfef-91c6-4111-982f-7c34e9607ala", enable_metric=>true, charset=>"UTF-8">, workers=>1, manage_template=>true, doc_as_upsert=>false, script_type=>"inline", script_lang=>"python", script_var_name=>"event", scripted_upsert=>false, retry_initial_interval=>2, retry_max_interval=>64, retry_on_conflict=>1, ilm_enabled=>"auto", ilm_pattern=>"now/d-000001", ilm_policy=>"logstash-policy", ecs_compatibility=>"disabled", action=>"index", ssl_certificate_verification=>true, sniffing=>false, sniffing_delay=>5, timeout=>60, pool_max=>100, pool_max_per_route=>100, resurrect_delay=>5, validate_after_inactivity=>10000, http_compression=>false}>
[2021-01-17T01:00:28,744][INFO ][logstash.outputs.elasticsearch][main] Elasticsearch pool URLs updated {:changes=>{:removed=>[], :added=>["http://127.0.0.1:9200/"]}}
[2021-01-17T01:00:28,949][WARN ][logstash.outputs.elasticsearch][main] Restored connection to ES instance {:url=>"http://127.0.0.1:9200/"}
[2021-01-17T01:00:29,006][INFO ][logstash.outputs.elasticsearch][main] ES Output version determined {:es_version=>7}
[2021-01-17T01:00:29,011][WARN ][logstash.outputs.elasticsearch][main] Detected a 6.x and above cluster: the 'type' event field won't be used to determine the document_type {:es_version=>7}
[2021-01-17T01:00:29,074][INFO ][logstash.outputs.elasticsearch][main] New Elasticsearch output {:class=>"LogStash::Output::ElasticSearch", :hosts=>["http://127.0.0.1:9200/"]}

=>1.08]
[2021-01-17T01:00:30,570][INFO ][logstash.javapipeline] [main] Pipeline started {"pipeline.id"=>"main"}
The stdin plugin is now waiting for input:
[2021-01-17T01:00:30,660][INFO ][logstash.agent] [main] Pipelines running {:count=>1, :running_pipelines=>[:main], :non_running_pipelines=>[]}
[2021-01-17T01:00:31,037][INFO ][logstash.agent] [main] Successfully started Logstash API endpoint {:port=>9600}
[2021-01-17T01:00:32,488][INFO ][logstash.inputs.jdbc] [main] [973eb8088e8891714435340a7a97f62bd28648cbe543472088cc78a32b9463ba] (0.020401s) SELECT * FROM android300
{
  "tag2" => "养成",
  "score" => 7.0999999904632568,
  "firm" => "网易游戏",
  "@timestamp" => 2021-01-16T17:00:32.576Z,
  "top" => 8,
  "tag3" => "回合制",
  "name" => "阴阳师",
  "tag1" => "卡牌",
  "download" => 2430089,
  "concern" => 1757736,
  "category" => "角色扮演",
  "@version" => "1"

  "tag2" => "卡牌养成",
  "score" => 7.5999999904632568,
  "firm" => "哔哩哔哩游戏",
  "@timestamp" => 2021-01-16T17:00:32.582Z,
```

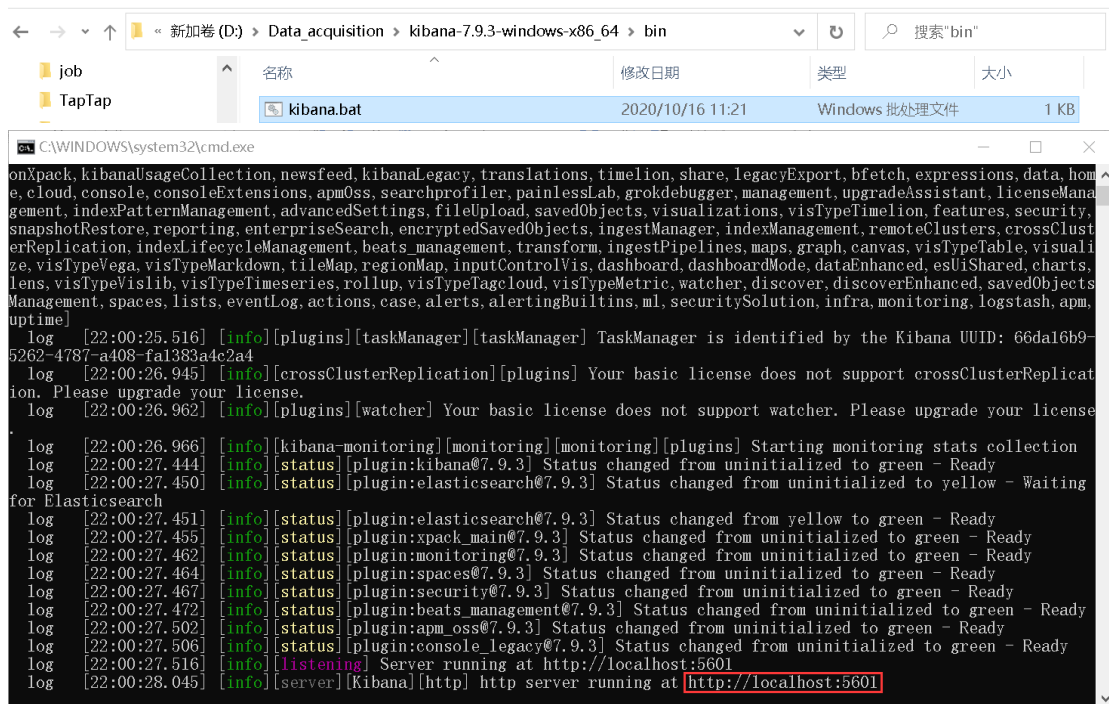
打开网页 localhost:9100，查看新建索引及数据。



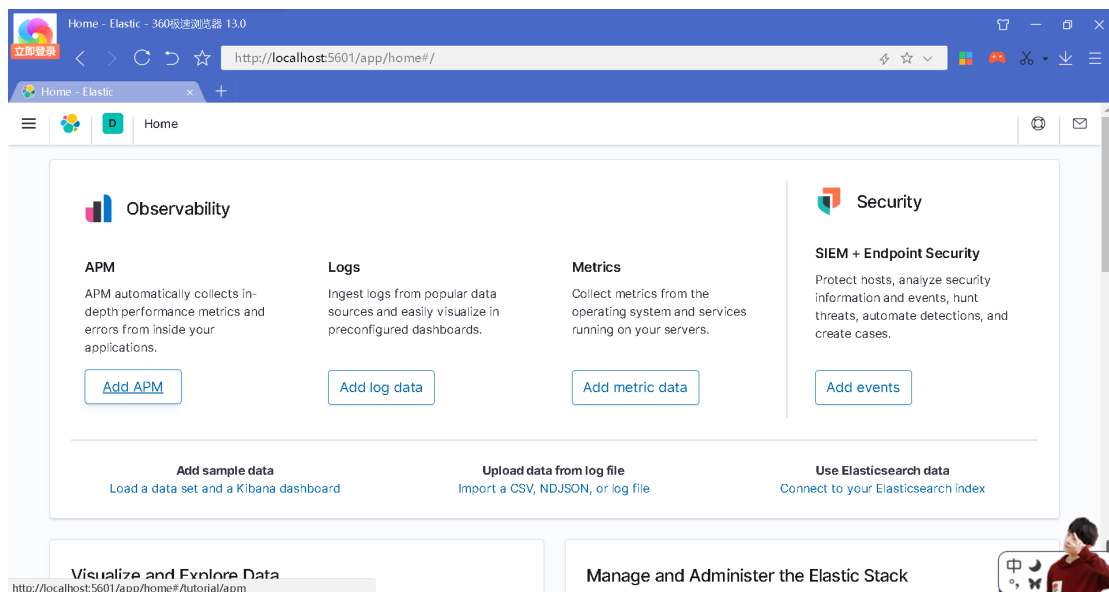


3.使用 kibana 进行数据可视化

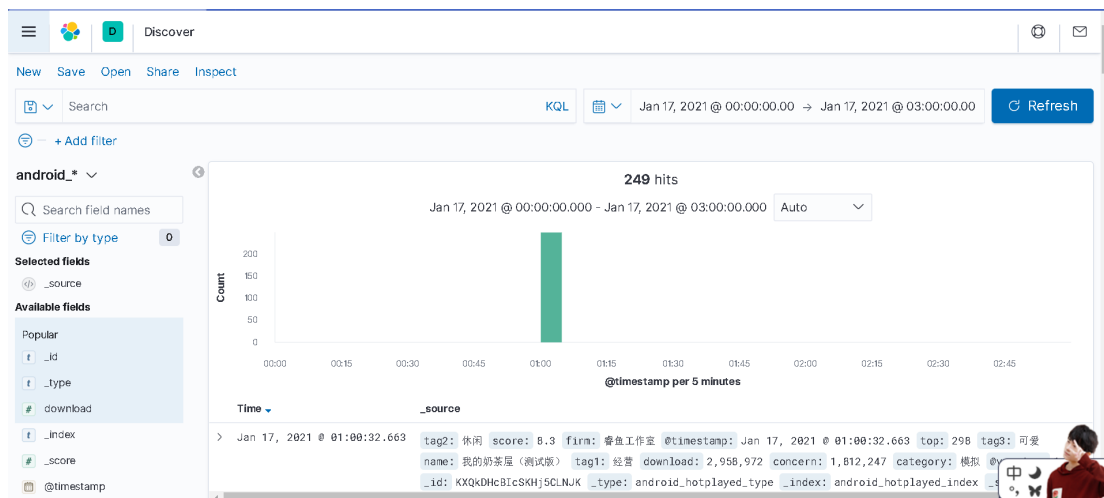
双击 kibana 的 bin 目录下的 kibana.bat 执行。



打开网页 <http://localhost:5601/>。



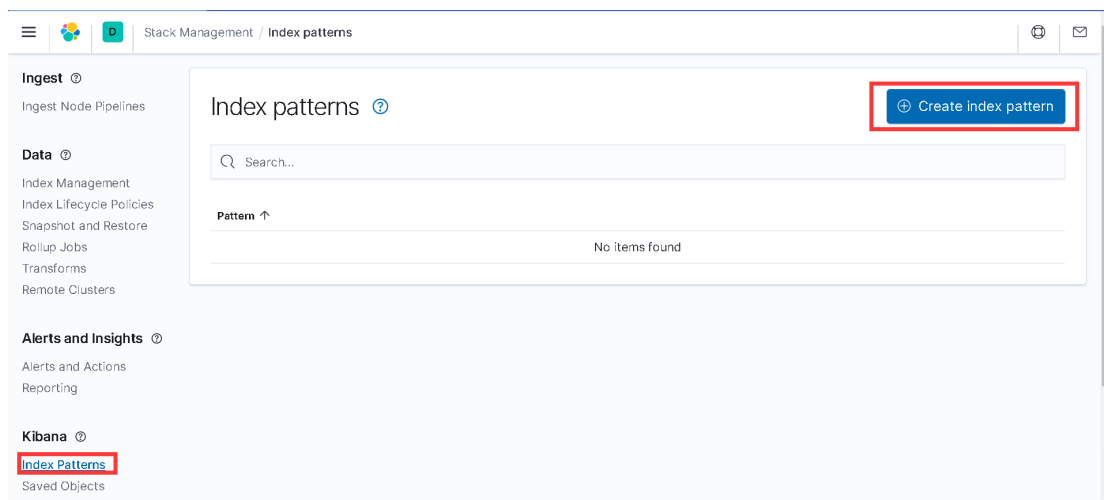
单击左侧菜单栏，进入 Kibana -> Discover。选择时间，可以看到有 1 月 17 日 1:00 上传的数据。



可以在左侧 Selected fields 选择要展示的内容。



单击左侧菜单栏，进入 Management -> Index Patterns。点击右上方 Create index pattern。



Create index pattern

An index pattern can match a single source, for example, `filebeat-4-3-22`, or **multiple** data sources, `filebeat-*`.
[Read documentation](#)

Step 1 of 2: Define index pattern

Index pattern name

android_hotplayed_*

Next step >

Use an asterisk (*) to match multiple indices. Spaces and the characters `\, /, ?, *, <, >, |` are not allowed.

☐ Include system and hidden indices

✓ Your index pattern matches 1 source.

android_hotplayed_index

Index

Rows per page: 10

Create index pattern

An index pattern can match a single source, for example, `filebeat-4-3-22`, or **multiple** data sources, `filebeat-*`.
[Read documentation](#)

Step 2 of 2: Configure settings

android_*

Select a primary time field for use with the global time filter.

Time field Refresh

@timestamp

> Show advanced options

< Back

Create index pattern

查看建立好的 index pattern。

Index patterns

Create index pattern

Search...

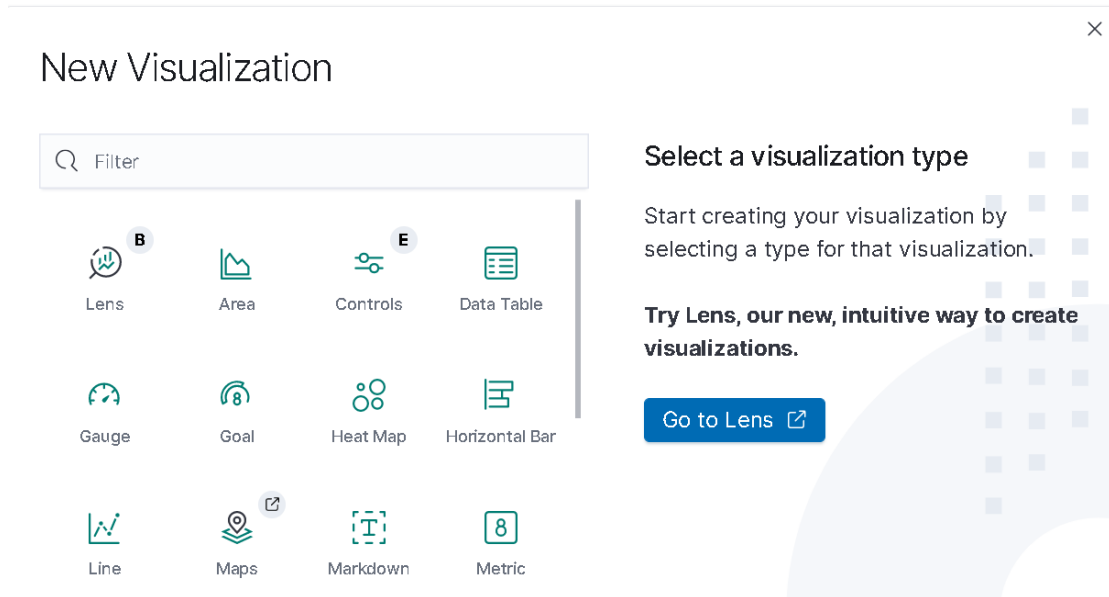
Pattern ↑

android_hotplayed_* Default

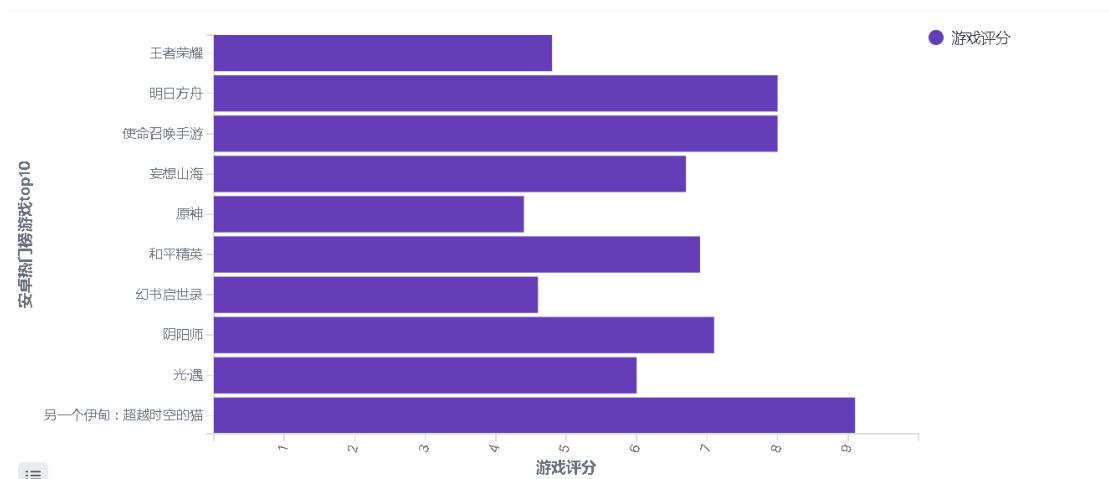
Rows per page: 10

< 1 >

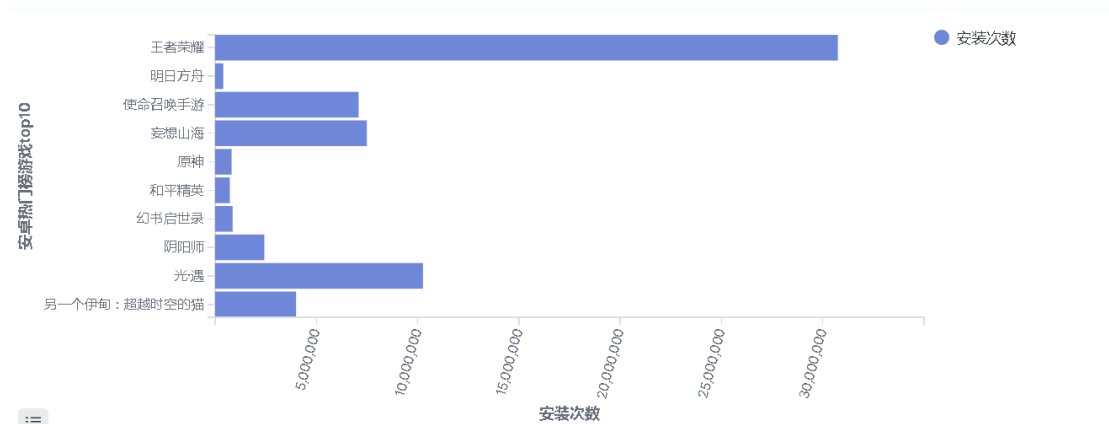
在左侧菜单选择 visualize，然后点击右边的+号，选择可视化的类型。



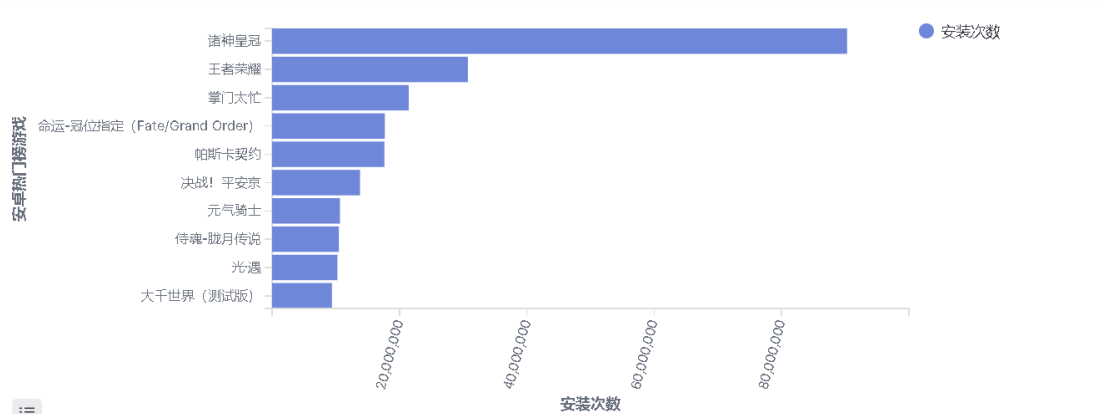
三、项目数据分析



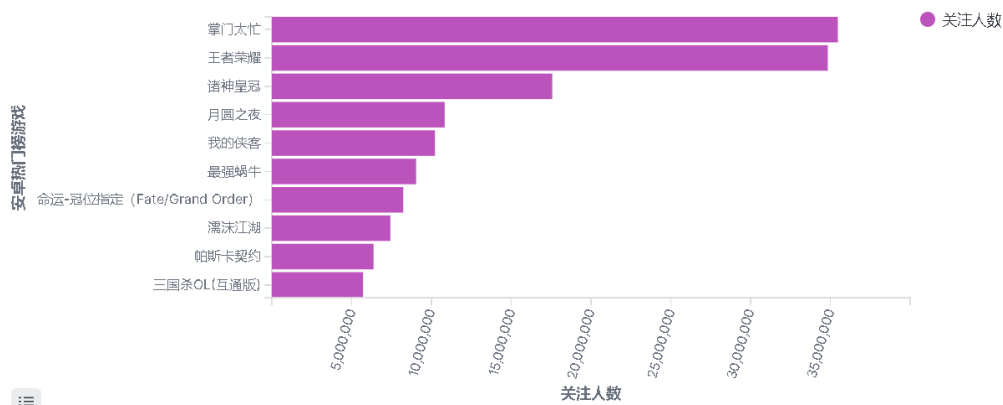
上图可以看出游戏评分和游戏热门程度并不成正比,但一般来说,玩家越多,评分也就越多,评分的可信度就越高。



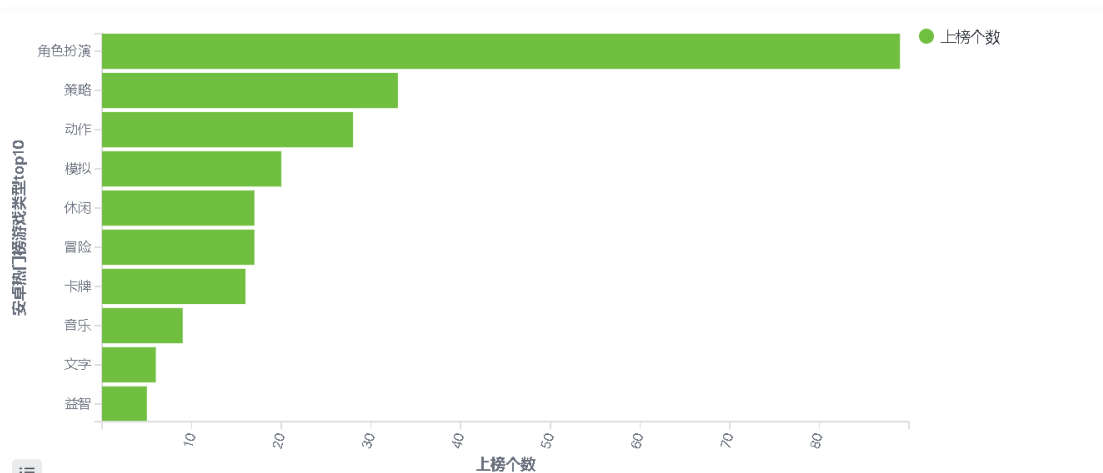
可以看出“王者荣耀”虽然评分不高，但在前十中安装次数遥遥领先其他九个游戏，更说明其火热程度。



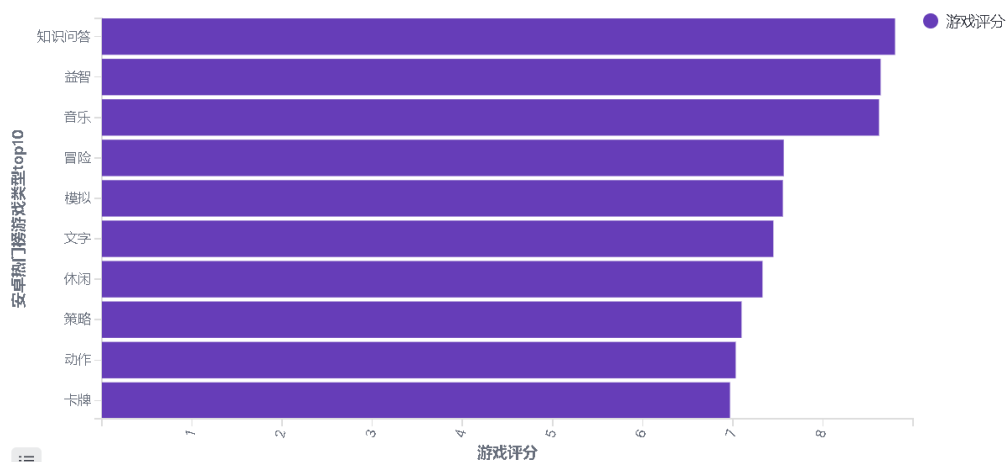
“诸神皇冠”的安装次数近 8 万 5，遥遥领先排在第二的“王者荣耀”。



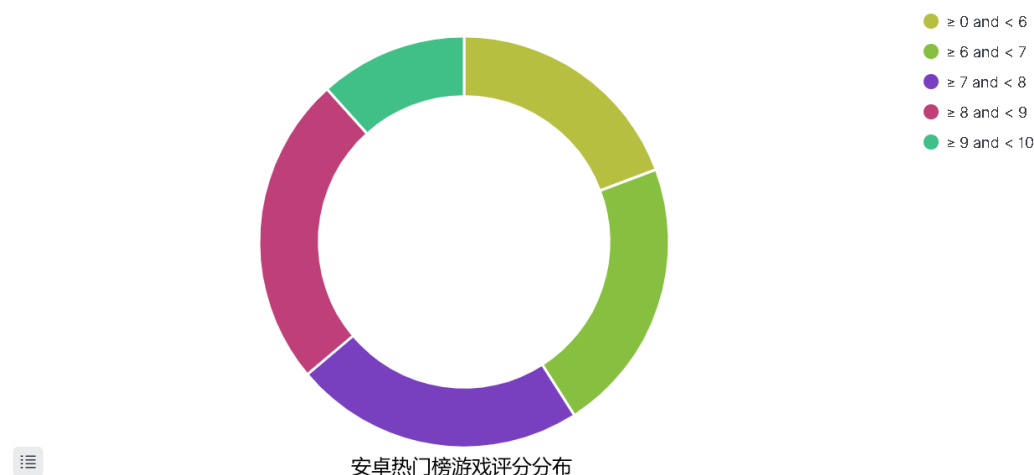
“掌门太忙”、“王者荣耀”以近 3.5 万的关注人数占领 top2，其次是“诸神皇冠”。



“角色扮演”以绝对性的优势作为上榜游戏类型的第一名，其次是“策略”和“动作”。



但从上图可以发现，“角色扮演”并未排进游戏类型评分 top10，而“策略”和“动作”以 7 分左右挤入 top10 的尾巴，“知识问答”却能以近 9 分的高分占据第一，说明上榜次数和玩家对其的评价并无关联。

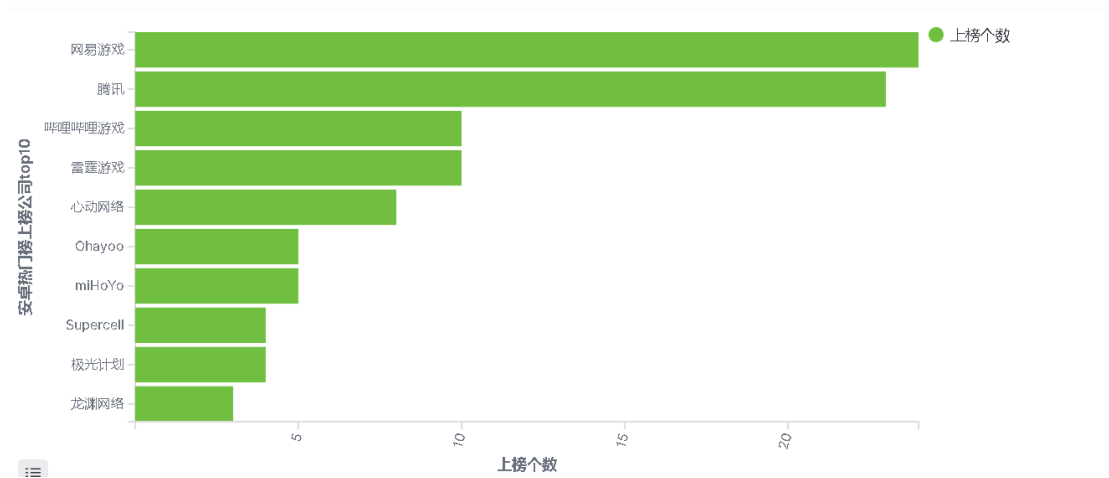


上图可以看出，安卓热门榜游戏评分分布在 0-6、6-7、7-8、8-9 的占比比较均匀，评分在 9-10 分之间的游戏占据得也不少。



安卓热门榜游戏标签统计 - Count

可以看到在热门游戏中二次元, 单机, 养成, 武侠, 养成, 策略等等为高频标签, 代表游戏如明日方舟等, 具备以上一些标签的游戏容易受欢迎。



四、项目总结

本次项目是基于对游戏的热情决定分析 TapTap 排行榜的数据。在整个数据获取、上传、清洗、建立索引、可视化的过程中, 花费时间最长的其实是数据获取。

因为之前只接触过 c++ 和 java, 没有 python 编程的经验, 所以刚上手时还略有不适, 因为 python 的代码并没有类似 c++ 的固定格式要求, 更加灵活, 所以直接写还有点困难。在 b 站先学习了简单的利用 python 对静态网页的抓取之后, 算是入门了 python 爬虫。但由于 TapTap 网页是 ajax 异步加载的, 而且返回的 HTML 的数据是文本格式, 挤在一行的, 并不是规整的 json 格式, 所以在尝试了使用 BeautifulSoup 获取 HTML 里对应标签失败之后, 最后选择了易理解的 selenium 动态抓取。虽然 selenium 动态抓取较为耗时、占内存, 但也相对更安全。最初版本的代码虽然可以运行, 但无法获得 300 条的数据, 且获取 150 条数据耗时最高可达 40 分钟, 极度吃网速和占内存, 所以后续又进行了改进。总的来说, 通过本次项目我了解了 python 爬网页的基本类别, 如静态、动态网页, 也掌握了一些 requests、beautifulsoup、selenium 以及 time、lock、threading 等库的基本常用方法。

在使用 Datax 上传 csv 文件到 mysql 过程中, 配置 json 文件是最核心的点, 代码的整体框架比较好理解, 核心是 reader 和 writer 的配置, 建议结合网上实例

再参考官方文档配置。运行 json 文件时，还是推荐使用绝对路径，可能是因为没有配置环境变量，所以本人在运行时相对路径会报错，同时，路径要使用 “/” 符号，而不是 “\”，因为 “\” 后面加字母都是转义字符，会报错。特别要检查是否有多余的括号或者分号，避免因格式导致错误。另一个需要注意的点是，在 mysql 中建表时，需添加 “ENGINE=InnoDB DEFAULT CHARSET=utf8”，保证数据库、表、数据的编码格式是一样的，这样在 mysql 中查看数据时就不会出现中文显示为 “？” 的情况。

使用 Kettle 清洗数据是相对来说最为简单的一个环节。从个人使用的经验来说，kettle 是一个很强大的数据清洗传输软件，功能齐全，但是初次上手有点摸不着头脑，转换的选项过多，容易找不到最适合需求的流程，对于这一点，只有多使用、多百度，熟能生巧。

ELK 的使用过程中，最繁杂的步骤就是 ELK 及插件的安装，也是整个项目实施过程中出错最多的步骤。如：logstash 启动报错，因为路径中不能含有中文；配置 elasticsearch.yml 文件时，新增代码的结尾需要加空格。

logstash启动报错

1 回答

慕圣7145719

老哥你解决这个问题没 我也碰到了

  回复

2019-06-12

dq_half

这个是因为你的路劲里面含有中文字符识别不了,你改成英文就好了

回复

2019-08-27

运行elasticsearch.bat出错

因为几乎没人回答这个问题，我是在某个评论中看到的

在配置elasticsearch.yml文件时 （我是在文件最后加入的下面的代码，在：后面要加个空格，不然运行会出错o(╥﹏╥)o）

```
http.cors.enabled: true
http.cors.allow-origin: "*"
http.cors.allow-methods: OPTIONS,HEAD,GET,POST,PUT,DELETE
http.cors.allow-headers: "X-Requested-With,Content-Type,Content-Length,X-User"
```

使用dataX收获的教训

首先是安装dataX，安装非常简单，只需下载对应的压缩包即可。

下载地址：<https://github.com/alibaba/DataX>

首先我出现的第一个错误：命令提示符界面出现乱码。

解决方法：

可以在cmd中输入：CHCP 65001，然后进行后续操作。

然后我遇到的最大的一个错误：

经DataX智能分析,该任务最可能的错误原因是:
com.alibaba.datax.common.exception.DataXException: Code:[Common-00], Describe:[您提供的配置文件存在错误信息，请检查您的作业配置.] - 配置信息错误。您提供的配置信息不是合法的JSON格式: unclosed string : *, 请按照标准json格式提供配置信息。

然后我百思不得其解，问了几个同学也没有结果，最后发现错误，真是被自己搞笑了。原来是路径写错了，比如该路径 D:/datax/datax/job/data1.csv

而我硬写成了 D:\datax\datax\job\data1.csv

经过上面的教训我终于知道了一般 \后面加字母都是转义字符，下次肯定不会弄乱了。

五、附录

（一）软件环境

软件		版本配置	运行系统
Java		1.8.0_271	windows
Python		3.9.1	
Datax		3.0	
Kettle		7.1	
ELK	Elasticsearch	7.9.3	
	Logstash	7.9.3	
	Kibana	7.9.3	
Mysql	Mysql	5.7.32	
	Mysql 驱动	5.7.32	

（二）多线程异步爬虫代码

安卓热玩榜前 150

```
from selenium import webdriver

import time

import pandas as pd

import threading
```

```
toplist = [0] * 150 # 游戏排名
namelist = [0] * 150 # 游戏名称
firmlist = [0] * 150 # 厂商名称
scorelist = [0] * 150 # 游戏评分
categorylist = [0] * 150 # 游戏分类
taglist = [0] * 150 # 游戏标签
statisticslist = [0] * 150 # 数据(关注、下载、购买)
games = list()

# 配置, 防止“不安全”
option = webdriver.ChromeOptions()
option.add_experimental_option('useAutomationExtension', False)
option.add_experimental_option('excludeSwitches', ['enable-automation'])

lock = threading.Lock()

def gettable(n, driver): # 第 n 页
    # 获得游戏排名
    top_content = driver.find_elements_by_xpath('//*[@id="topList"]/div/span[2]')
    for i in range(0, 30):
        toplist[(n - 1) * 30 + i] = top_content[(n - 1) * 30 + i].text
    # 获得游戏名称
    name_content = driver.find_elements_by_xpath('//*[@id="topList"]/div/div[3]/a/h4')
    for i in range(0, 30):
        namelist[(n - 1) * 30 + i] = name_content[(n - 1) * 30 + i].text
    # 获得厂商名称
    firm_content = driver.find_elements_by_xpath('//*[@id="topList"]/div/div[3]/p[1]/a')
    for i in range(0, 30):
        firmlist[(n - 1) * 30 + i] = firm_content[(n - 1) * 30 + i].text
```

```
# 获得游戏评分

score_content = driver.find_elements_by_xpath('//*[@id="topList"]/div/div[3]/div[1]/p/span')

for i in range(0, 30):

    scorelist[(n - 1) * 30 + i] = score_content[(n - 1) * 30 + i].text

# 获得游戏分类

category_content = driver.find_elements_by_xpath('//*[@id="topList"]/div/div[3]/div[3]/a')

for i in range(0, 30):

    categorylist[(n - 1) * 30 + i] = category_content[(n - 1) * 30 + i].text

# 获得游戏标签

tag_content = driver.find_elements_by_xpath('//*[@id="topList"]/div/div[3]/div[2]')

for i in range(0, 30):

    taglist[(n - 1) * 30 + i] = tag_content[(n - 1) * 30 + i].text.replace('\n', ',')

def getpage(n, driver):

    for i in range(1, n):

        search_button = driver.find_element_by_xpath('//*[@id="page-top"]/section[5]/button')

        search_button.click()

        time.sleep(3)

def getstatistics(n, driver): # 获得第 n 页的游戏数据

    links = driver.find_elements_by_xpath('//*[@id="topList"]/div/div[3]/a')

    for i in range(0, 6): # 分为六组，每五个为一组获取

        driver2 = webdriver.Chrome(chrome_options=option)

        driver2.get(links[(n - 1) * 30 + i * 5].get_attribute('href'))

        driver3 = webdriver.Chrome(chrome_options=option)

        driver3.get(links[(n - 1) * 30 + i * 5 + 1].get_attribute('href'))

        driver4 = webdriver.Chrome(chrome_options=option)

        driver4.get(links[(n - 1) * 30 + i * 5 + 2].get_attribute('href'))
```

```

driver5 = webdriver.Chrome(chrome_options=option)

driver5.get(links[(n - 1) * 30 + i * 5 + 3].get_attribute('href'))

driver6 = webdriver.Chrome(chrome_options=option)

driver6.get(links[(n - 1) * 30 + i * 5 + 4].get_attribute('href'))

# 获得下载次数

statistics_content =

driver2.find_elements_by_xpath('//*[@id="js-nav-sidebar-main"]/div[1]/div/div/section[1]/div[1]/div[2]/div[2]/div[1]/div/p')

for statistics in statistics_content:

    statisticslist[(n - 1) * 30 + (i - 1) * 5] = statistics.text

driver2.close()

statistics_content =

driver3.find_elements_by_xpath('//*[@id="js-nav-sidebar-main"]/div[1]/div/div/section[1]/div[1]/div[2]/div[2]/div[1]/div/p')

for statistics in statistics_content:

    statisticslist[(n - 1) * 30 + (i - 1) * 5 + 1] = statistics.text

driver3.close()

statistics_content =

driver4.find_elements_by_xpath('//*[@id="js-nav-sidebar-main"]/div[1]/div/div/section[1]/div[1]/div[2]/div[2]/div[1]/div/p')

for statistics in statistics_content:

    statisticslist[(n - 1) * 30 + (i - 1) * 5 + 2] = statistics.text

driver4.close()

statistics_content =

driver5.find_elements_by_xpath('//*[@id="js-nav-sidebar-main"]/div[1]/div/div/section[1]/div[1]/div[2]/div[2]/div[1]/div/p')

for statistics in statistics_content:

    statisticslist[(n - 1) * 30 + (i - 1) * 5 + 3] = statistics.text

driver5.close()

statistics_content =

```

```
driver6.find_elements_by_xpath('//*[@id="js-nav-sidebar-main"]/div[1]/div/div/section[1]/div[1]/div[2]/div[2]/div[1]/div/p')
```

```
    for statistics in statistics_content:
```

```
        statisticslist[(n - 1) * 30 + (i - 1) * 5 + 4] = statistics.text
```

```
    driver6.close()
```

```
class getpage1(threading.Thread):
```

```
    def run(self):
```

```
        driver01 = webdriver.Chrome(chrome_options=option)
```

```
        driver01.get('https://www.taptap.com/top/played')
```

```
        gettable(1, driver01)
```

```
        getstatistics(1, driver01)
```

```
class getpage2(threading.Thread):
```

```
    def run(self):
```

```
        driver02 = webdriver.Chrome(chrome_options=option)
```

```
        driver02.get('https://www.taptap.com/top/played')
```

```
        getpage(2, driver02)
```

```
        gettable(2, driver02)
```

```
        getstatistics(2, driver02)
```

```
class getpage3(threading.Thread):
```

```
    def run(self):
```

```
        driver03 = webdriver.Chrome(chrome_options=option)
```

```
        driver03.get('https://www.taptap.com/top/played')
```

```
        getpage(3, driver03)
```

```
        gettable(3, driver03)
```

```
getstatistics(3, driver03)
```

```
class getpage4(threading.Thread):
```

```
    def run(self):
```

```
        driver04 = webdriver.Chrome(chrome_options=option)
```

```
        driver04.get('https://www.taptap.com/top/played')
```

```
        getpage(4, driver04)
```

```
        gettable(4, driver04)
```

```
        getstatistics(4, driver04)
```

```
class getpage5(threading.Thread):
```

```
    def run(self):
```

```
        driver05 = webdriver.Chrome(chrome_options=option)
```

```
        driver05.get('https://www.taptap.com/top/played')
```

```
        getpage(5, driver05)
```

```
        gettable(5, driver05)
```

```
        getstatistics(5, driver05)
```

```
def multi_thread():
```

```
    t1 = getpage1()
```

```
    t2 = getpage2()
```

```
    t3 = getpage3()
```

```
    t4 = getpage4()
```

```
    t5 = getpage5()
```

```
    t1.start()
```

```
    t2.start()
```

```
t3.start()

t4.start()

t5.start()


t1.join()

t2.join()

t3.join()

t4.join()

t5.join()


if __name__ == '__main__':

    lock.acquire()

    multi_thread()

    lock.release()

    for top, name, firm, score, category, tag, statistics in zip(toplist, namelist, firmlist, scorelist,
categorylist, taglist, statisticslist):

        game = {

            "游戏排名": top,

            "游戏名称": name,

            "游戏厂商": firm,

            "游戏评分": score,

            "游戏分类": category,

            "游戏标签": tag,

            "游戏数据": statistics

        }

        games.append(game)

        print(game)


test = pd.DataFrame(data=games)
```



```
test.to_csv('E:/TapTap/2021.1.6/Android_HotPlayed.csv', index=0)
```

安卓热玩榜后 150

```
from selenium import webdriver
```

```
import time
```

```
import pandas as pd
```

```
import threading
```

```
toplist = [0] * 150 # 游戏排名
```

```
namelist = [0] * 150 # 游戏名称
```

```
firmlist = [0] * 150 # 厂商名称
```

```
scorelist = [0] * 150 # 游戏评分
```

```
categorylist = [0] * 150 # 游戏分类
```

```
taglist = [0] * 150 # 游戏标签
```

```
statisticslist = [0] * 150 # 数据(关注、下载、购买)
```

```
games = list()
```

```
# 配置，防止“不安全”
```

```
option = webdriver.ChromeOptions()
```

```
option.add_experimental_option('useAutomationExtension', False)
```

```
option.add_experimental_option('excludeSwitches', ['enable-automation'])
```

```
lock = threading.Lock()
```

```
def gettable(n, driver): # 第 n 页
```

```
    # 获得游戏排名
```

```
    top_content = driver.find_elements_by_xpath('//*[@id="topList"]/div/span[2])
```

```
    for i in range(0, 30):
```

```
        toplist[((n - 1) * 30 + i)-150] = top_content[(n - 1) * 30 + i].text
```

```

# 获得游戏名称

name_content = driver.find_elements_by_xpath('//*[@id="topList"]/div/div[3]/a/h4')

for i in range(0, 30):

    namelist[((n - 1) * 30 + i)-150] = name_content[(n - 1) * 30 + i].text

# 获得厂商名称

firm_content = driver.find_elements_by_xpath('//*[@id="topList"]/div/div[3]/p[1]/a')

for i in range(0, 30):

    firmlist[((n - 1) * 30 + i)-150] = firm_content[(n - 1) * 30 + i].text

# 获得游戏评分

score_content = driver.find_elements_by_xpath('//*[@id="topList"]/div/div[3]/div[1]/p/span')

for i in range(0, 30):

    scorelist[((n - 1) * 30 + i)-150] = score_content[(n - 1) * 30 + i].text

# 获得游戏分类

category_content = driver.find_elements_by_xpath('//*[@id="topList"]/div/div[3]/div[3]/a')

for i in range(0, 30):

    categorylist[((n - 1) * 30 + i)-150] = category_content[(n - 1) * 30 + i].text

# 获得游戏标签

tag_content = driver.find_elements_by_xpath('//*[@id="topList"]/div/div[3]/div[2]')

for i in range(0, 30):

    taglist[((n - 1) * 30 + i)-150] = tag_content[(n - 1) * 30 + i].text.replace('\n', ',')

def getpage(n, driver):

    for i in range(1, n):

        search_button = driver.find_element_by_xpath('//*[@id="page-top"]/section[5]/button')

        search_button.click()

        time.sleep(3)

def getstatistics(n, driver): # 获得第 n 页的游戏数据

```

```

links = driver.find_elements_by_xpath('//*[@id="topList"]/div/div[3]/a')

for i in range(0, 6): # 分为六组，每五个为一组获取

    driver2 = webdriver.Chrome(chrome_options=option)

    driver2.get(links[(n - 1) * 30 + i * 5].get_attribute('href'))

    driver3 = webdriver.Chrome(chrome_options=option)

    driver3.get(links[(n - 1) * 30 + i * 5 + 1].get_attribute('href'))

    driver4 = webdriver.Chrome(chrome_options=option)

    driver4.get(links[(n - 1) * 30 + i * 5 + 2].get_attribute('href'))

    driver5 = webdriver.Chrome(chrome_options=option)

    driver5.get(links[(n - 1) * 30 + i * 5 + 3].get_attribute('href'))

    driver6 = webdriver.Chrome(chrome_options=option)

    driver6.get(links[(n - 1) * 30 + i * 5 + 4].get_attribute('href'))

    # 获得下载次数

    statistics_content =

driver2.find_elements_by_xpath('//*[@id="js-nav-sidebar-main"]/div[1]/div/div/section[1]/div[1]/div[2]/div[2]/div[1]/div/p')

    for statistics in statistics_content:

        statisticslist[((n - 1) * 30 + (i - 1) * 5)-150] = statistics.text

    driver2.close()

    statistics_content =

driver3.find_elements_by_xpath('//*[@id="js-nav-sidebar-main"]/div[1]/div/div/section[1]/div[1]/div[2]/div[2]/div[1]/div/p')

    for statistics in statistics_content:

        statisticslist[((n - 1) * 30 + (i - 1) * 5 + 1)-150] = statistics.text

    driver3.close()

    statistics_content =

driver4.find_elements_by_xpath('//*[@id="js-nav-sidebar-main"]/div[1]/div/div/section[1]/div[1]/div[2]/div[2]/div[1]/div/p')

    for statistics in statistics_content:

        statisticslist[((n - 1) * 30 + (i - 1) * 5 + 2)-150] = statistics.text

```

```

driver4.close()

statistics_content =
driver5.find_elements_by_xpath('//*[@id="js-nav-sidebar-main"]/div[1]/div/div/section[1]/div[1]/div[2]/div[2]/div[1]/div/p')

for statistics in statistics_content:
    statisticslist[((n - 1) * 30 + (i - 1) * 5 + 3)-150] = statistics.text

driver5.close()

statistics_content =
driver6.find_elements_by_xpath('//*[@id="js-nav-sidebar-main"]/div[1]/div/div/section[1]/div[1]/div[2]/div[2]/div[1]/div/p')

for statistics in statistics_content:
    statisticslist[((n - 1) * 30 + (i - 1) * 5 + 4)-150] = statistics.text

driver6.close()

```

```

class getpage6(threading.Thread):

```

```

    def run(self):

```

```

        driver06 = webdriver.Chrome(chrome_options=option)

        driver06.get('https://www.taptap.com/top/played')

        getpage(6, driver06)

        gettable(6, driver06)

        getstatistics(6, driver06)

```

```

class getpage7(threading.Thread):

```

```

    def run(self):

```

```

        driver07 = webdriver.Chrome(chrome_options=option)

        driver07.get('https://www.taptap.com/top/played')

        getpage(7, driver07)

        gettable(7, driver07)

```

```
getstatistics(7, driver07)
```

```
class getpage8(threading.Thread):
```

```
    def run(self):
```

```
        driver08 = webdriver.Chrome(chrome_options=option)
```

```
        driver08.get('https://www.taptap.com/top/played')
```

```
        getpage(8, driver08)
```

```
        gettable(8, driver08)
```

```
        getstatistics(8, driver08)
```

```
class getpage9(threading.Thread):
```

```
    def run(self):
```

```
        driver09 = webdriver.Chrome(chrome_options=option)
```

```
        driver09.get('https://www.taptap.com/top/played')
```

```
        getpage(9, driver09)
```

```
        gettable(9, driver09)
```

```
        getstatistics(9, driver09)
```

```
class getpage10(threading.Thread):
```

```
    def run(self):
```

```
        driver10 = webdriver.Chrome(chrome_options=option)
```

```
        driver10.get('https://www.taptap.com/top/played')
```

```
        getpage(10, driver10)
```

```
        gettable(10, driver10)
```

```
        getstatistics(10, driver10)
```

```
def multi_thread():  
    t6 = getpage6()  
    t7 = getpage7()  
    t8 = getpage8()  
    t9 = getpage9()  
    t10 = getpage10()  
  
    t6.start()  
    t7.start()  
    t8.start()  
    t9.start()  
    t10.start()  
  
    t6.join()  
    t7.join()  
    t8.join()  
    t9.join()  
    t10.join()  
  
if __name__ == '__main__':  
    lock.acquire()  
    multi_thread()  
    lock.release()  
  
    for top, name, firm, score, category, tag, statistics in zip(toplist, namelist, firmlist, scorelist,  
categorylist, taglist, statisticslist):  
        game = {  
            "游戏排名": top,  
            "游戏名称": name,  
            "游戏厂商": firm,
```

```
        "游戏评分": score,

        "游戏分类": category,

        "游戏标签": tag,

        "游戏数据": statistics

    }

    games.append(game)

    print(game)

test = pd.DataFrame(data=games)

test.to_csv('E:/TapTap/2021.1.6/Android_HotPlayed.csv', encoding="utf_8_sig", index=False,
mode='a+', header=False)
```

（二）json 文件

```
{
  "job": {
    "content": [
      {
        "reader": {
          "name": "txtfilereader",
          "parameter": {
            "path": ["E:/TapTap/Android_HotPlayed.csv"],
            "encoding": "GBK",
            "fieldDelimiter": ",",
            "skipHeader": "true",
            "column": [
              {
                "index": 0,
                "type": "string"
              },
              {
                "index": 1,
                "type": "string"
              },
              {
                "index": 2,
                "type": "string"
              }
            ]
          }
        }
      }
    ]
  }
}
```

```

        "index": 3,
        "type": "string"
    },
    {
        "index": 4,
        "type": "string"
    },
    {
        "index": 5,
        "type": "string"
    },
    {
        "index": 6,
        "type": "string"
    }
]
},
"writer": {
    "name": "mysqlwriter",
    "parameter": {
        "column": [
            "top",
            "name",
            "firm",
            "score",
            "category",
            "tag",
            "statistics"
        ],
        "connection": [
            {
                "table": ["android_hotplayed"],
                "jdbcUrl":
"jdbc:mysql://localhost:3306/TapTap?useUnicode=true&characterEncoding=utf-8"
            }
        ],
        "password": "root",
        "username": "root",
        "preSql": ["delete from android_hotplayed"],
        "session": ["set session sql_mode='ANSI'"],
        "writeMode": "insert"
    }
}
}

```



```

    }
  ],
  "setting": {
    "speed": {
      "channel": "2"
    }
  }
}
}
}

```

(三) conf 文件

```

input {

  stdin {

  }

  jdbc {

    jdbc_connection_string =>
"jdbc:mysql://localhost:3306/taptap?characterEncoding=UTF-8&useSSL=false&autoReconnect=true"

    jdbc_user => "root"

    jdbc_password => "root"

    jdbc_driver_library => "D:\Data_acquisition\mysql\mysql-connector-java-5.1.38.jar"

    jdbc_driver_class => "com.mysql.jdbc.Driver"

    jdbc_default_timezone => "Asia/Shanghai"

    statement => "SELECT * FROM android300"

  }

}

output {

  elasticsearch {

    hosts => ["127.0.0.1:9200"]

    index => "android_hotplayed_index"

```

```
      document_type => "android_hotplayed_type"

      template_overwrite => true
    }

    stdout {
      codec => "rubydebug"
    }
  }
}
```